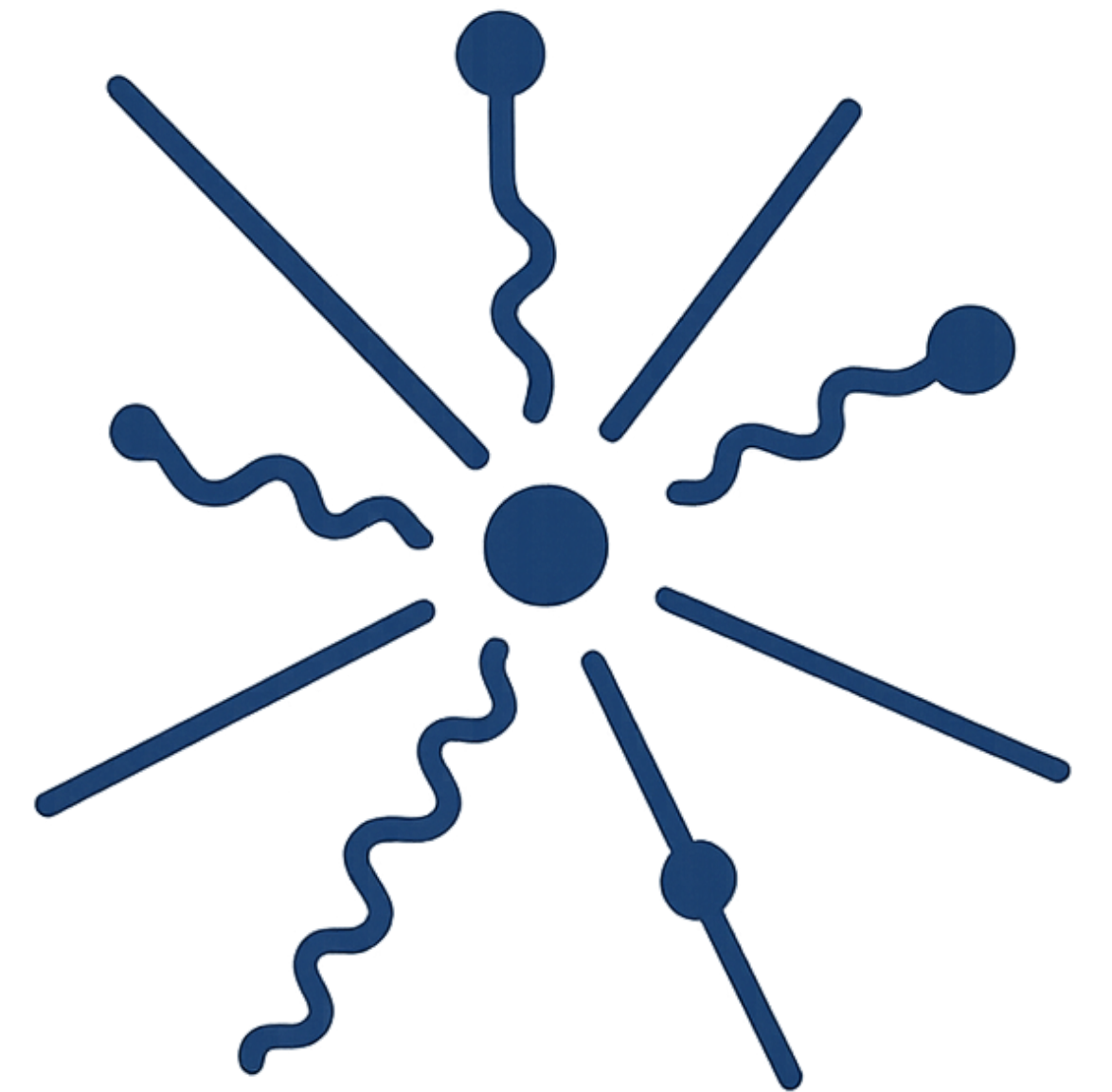
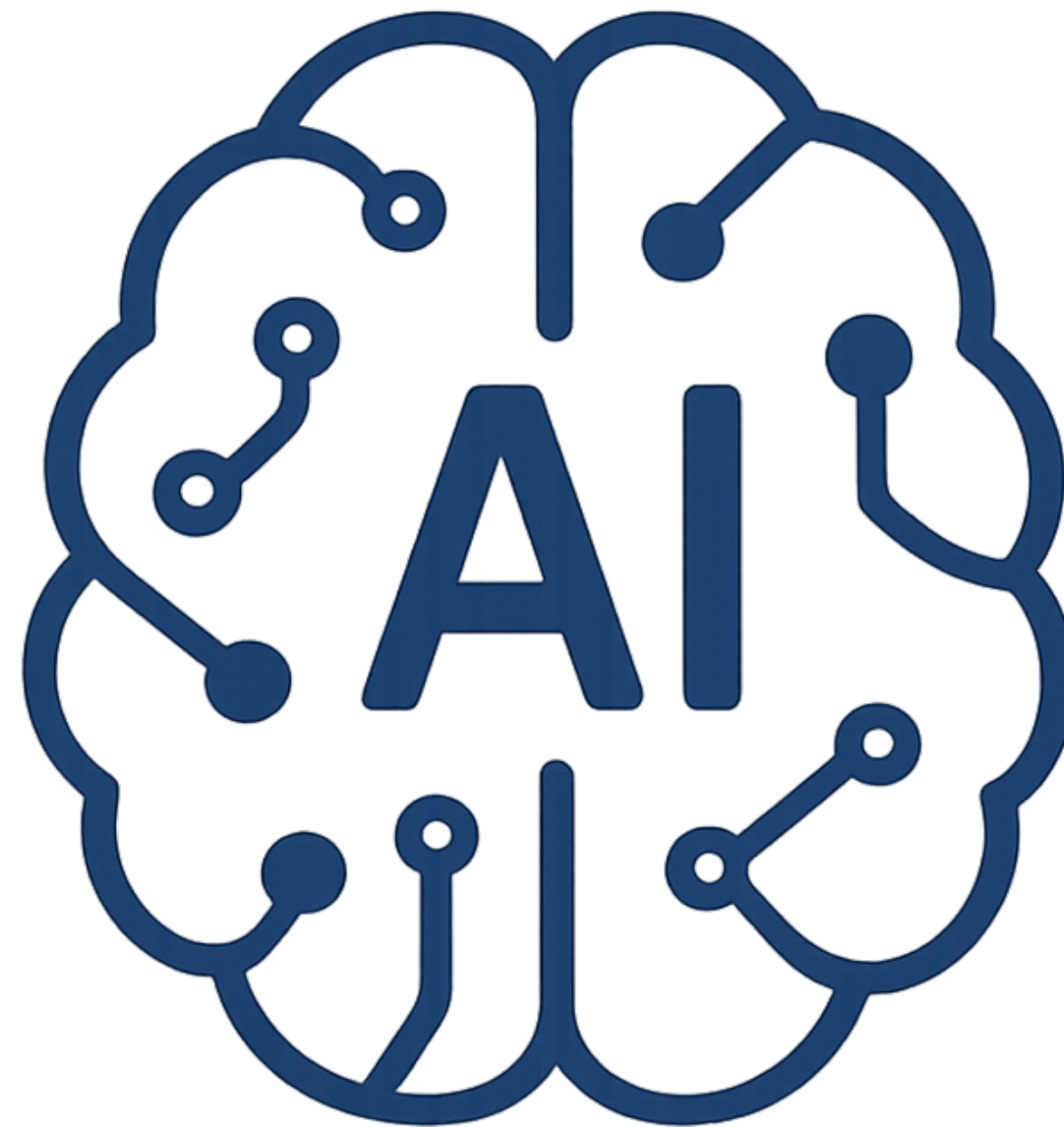
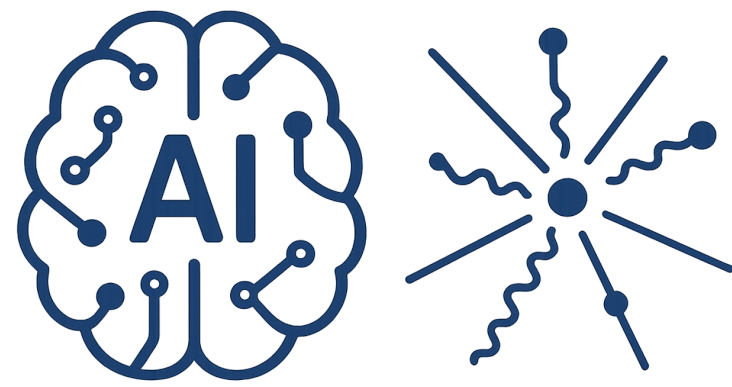


**Introduction to**

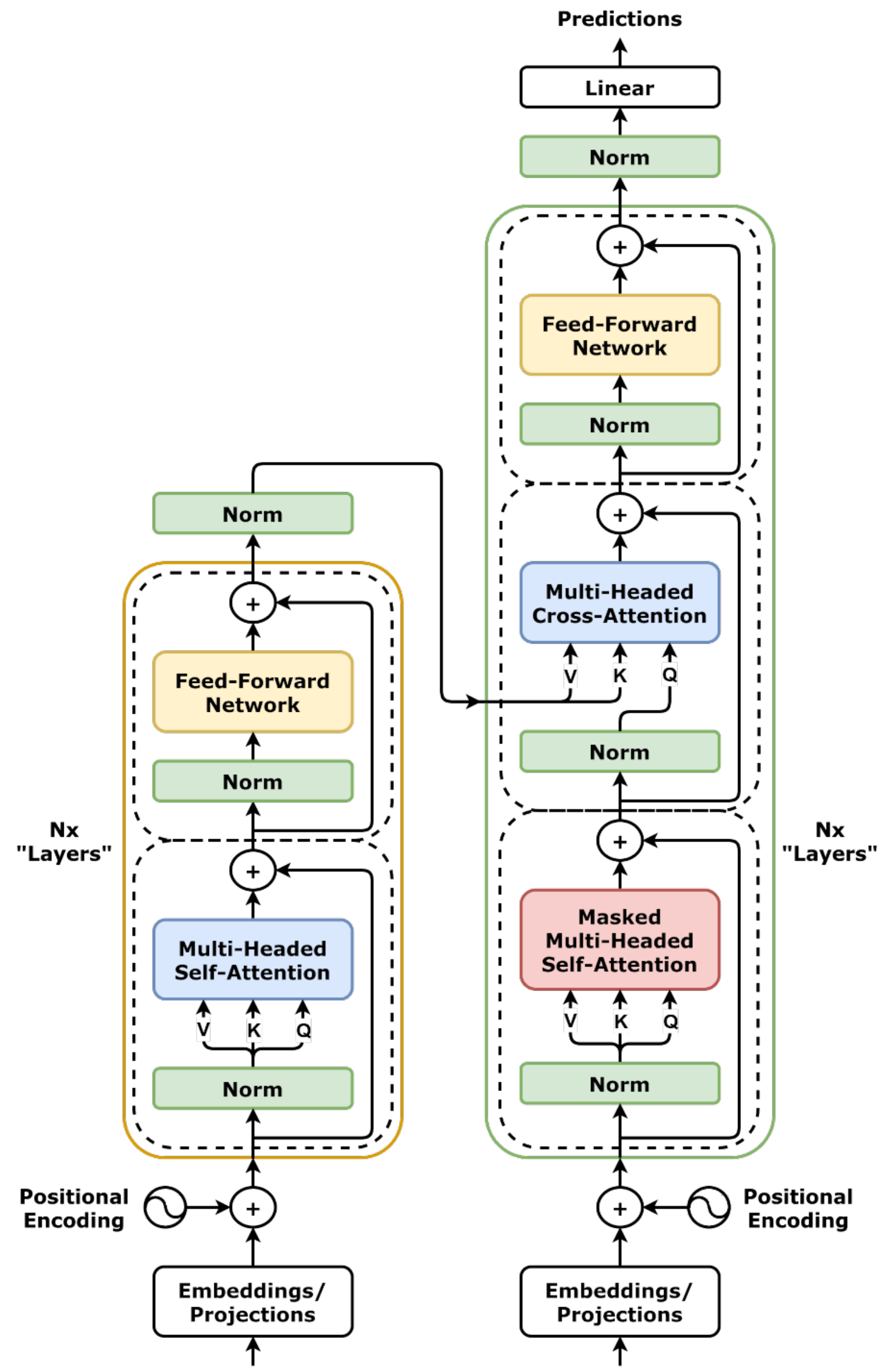
# **AI-Driven HEP**

**S. A. Fard - School of Physics (IPM)**

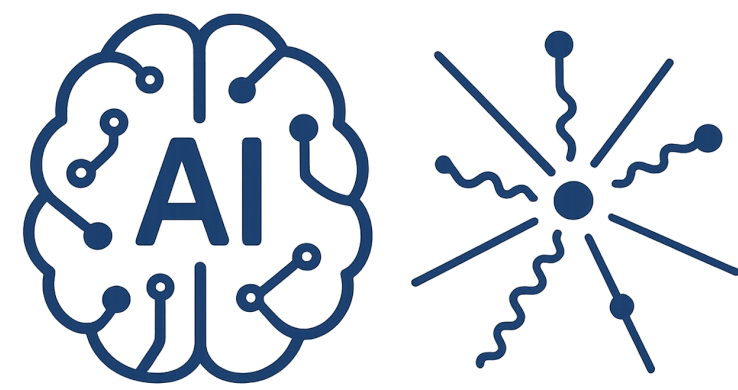




# AI-Driven HEP 19



## Session 19 Transformer



## AI-Driven HEP 19

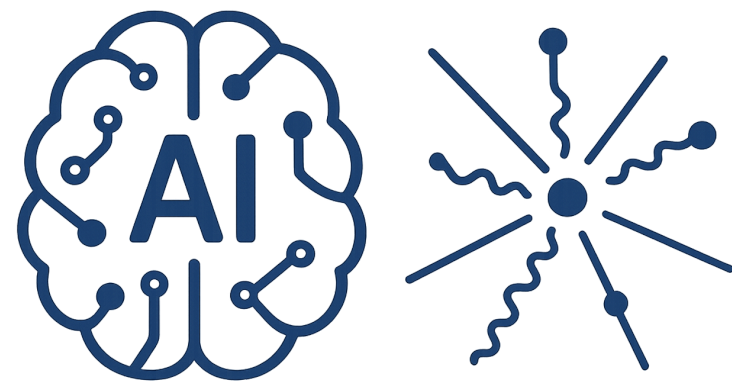
### Reference

- [1] Bishop, Christopher M., and Hugh Bishop. Deep learning: Foundations and concepts. Springer Nature, 2023.
- [2] By dvgodoy - <https://github.com/dvgodoy/dl-visuals/?tab=readme-ov-file>
- [3] <https://arxiv.org/abs/2404.16091>, Solving Key Challenges in Collider Physics with Foundation Models
- [4] <https://arxiv.org/pdf/2512.01324>, <https://github.com/DeepLearnPhysics/Panda>
- [5] Tarek Abouelfadl Mohamed, A. (2020). The LHC and the ATLAS Experiment. In: Measurement of Higgs Boson Production Cross Sections in the Diphoton Channel. Springer Theses. Springer, Cham

<https://physics.ipm.ac.ir/~ansarifard/>



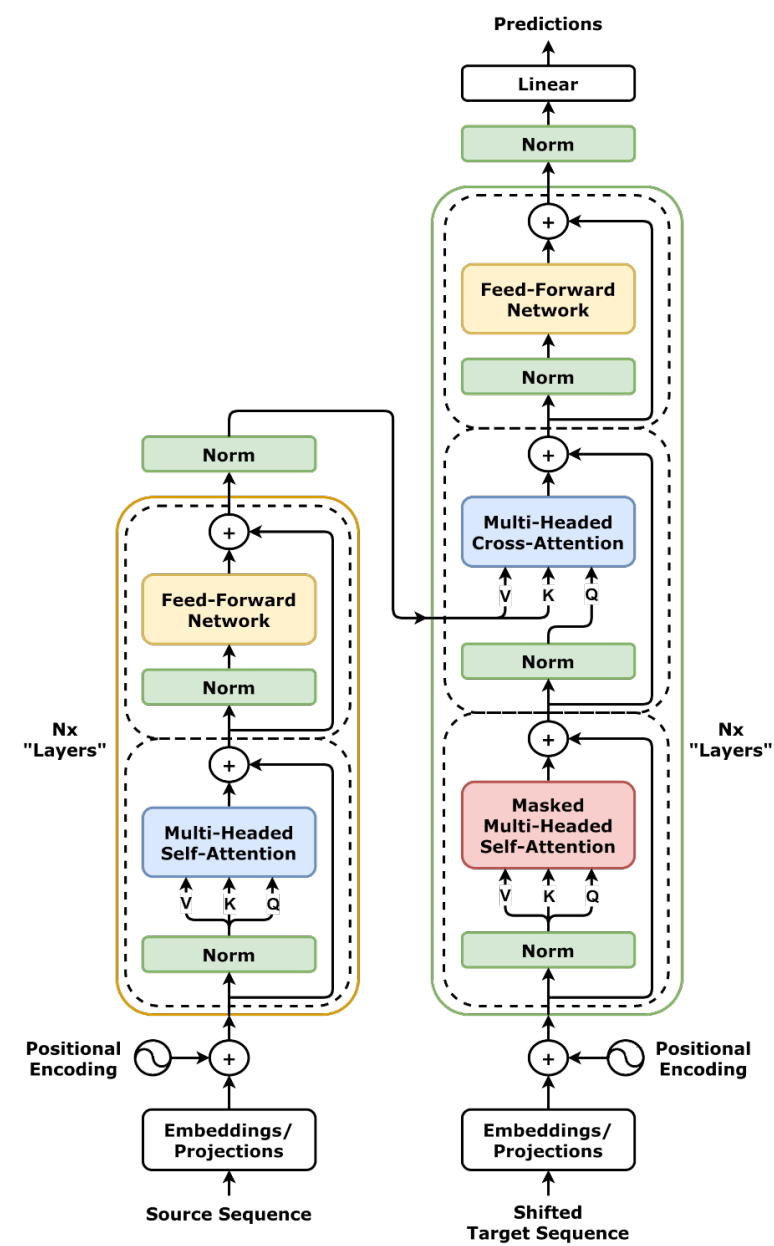




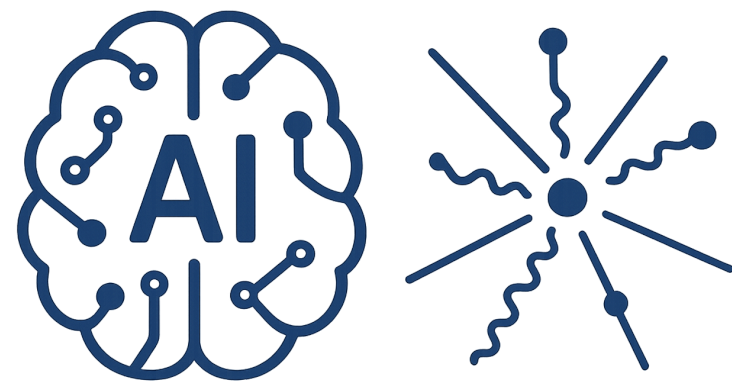
# Multimodal Transformers

Transformers as a general-purpose models

text, image, video,  
points cloud, signal







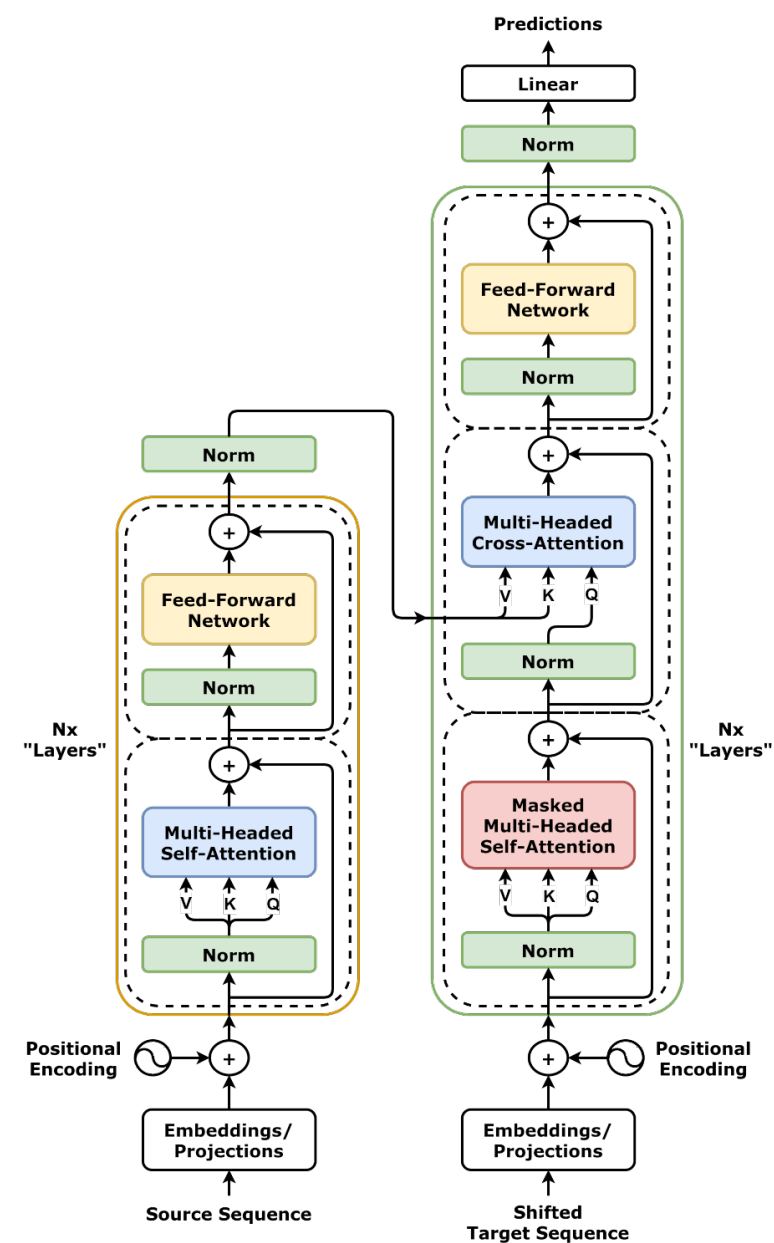
## Multimodal Transformers

Transformers as a general-purpose models

text, image, video,  
points cloud, signal

Transformers make multimodal computation  
relatively straightforward

convert non-text data into a sequence of vectors  
the transformer can process.



## VisionTransformer

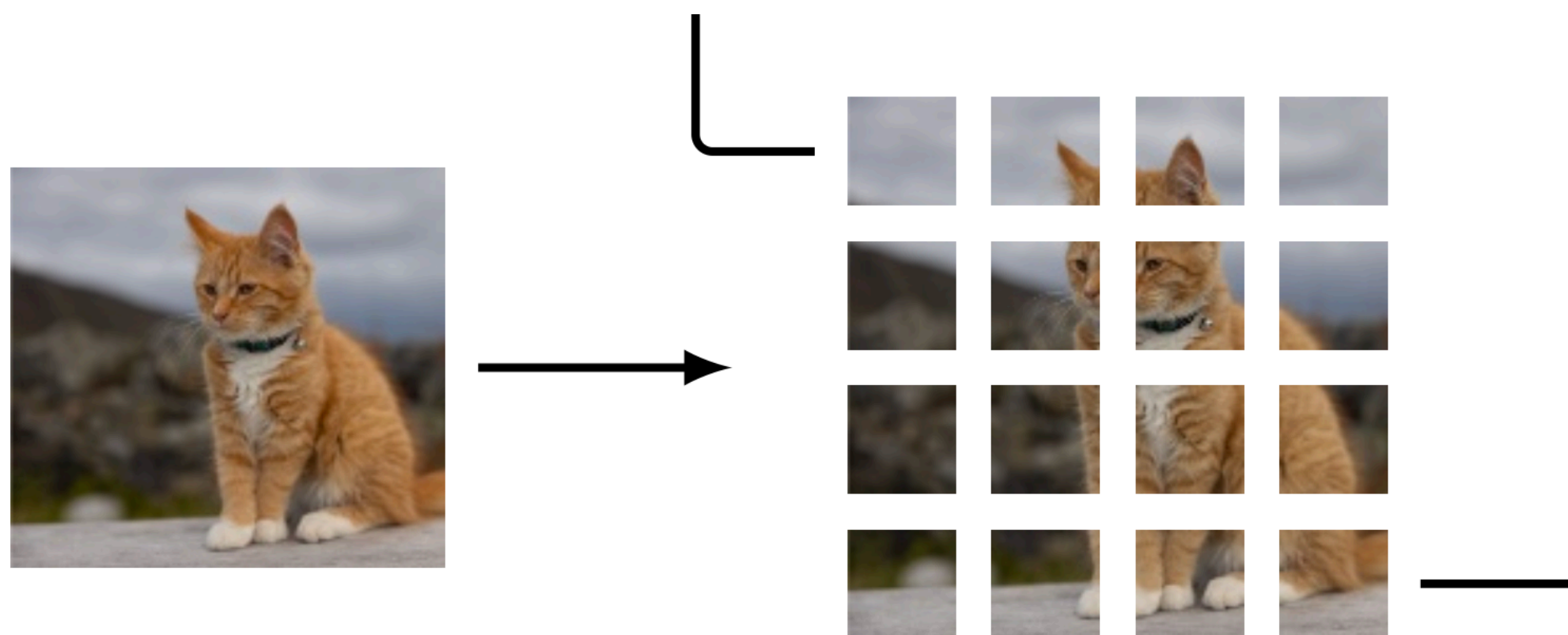
split images into non-overlapping patches

flattened into a one-dimensional vector

training : patch reconstructions in output

$$\mathbf{x}_p \in \mathbb{R}^{N \times (P^2 C)}$$

$$N = HW/P^2$$



## VisionTransformer

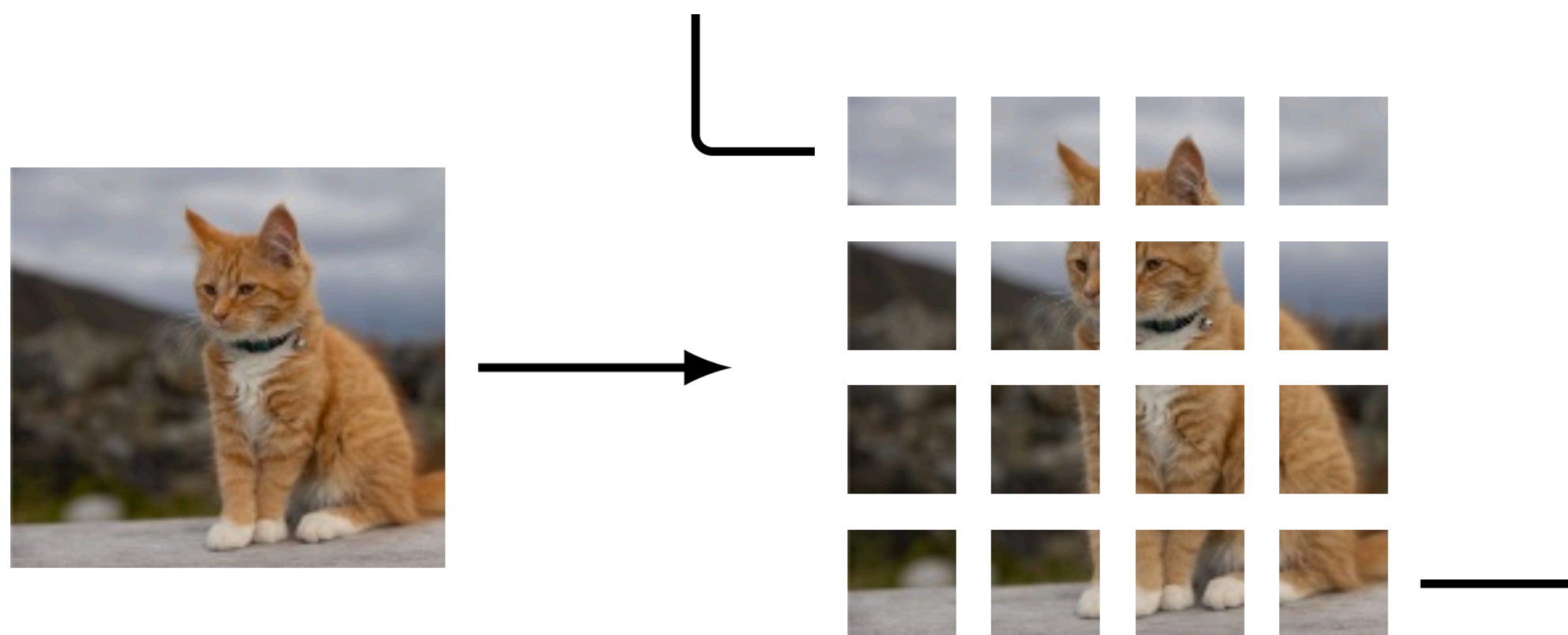
split images into non-overlapping patches

flattened into a one-dimensional vector

training : patch reconstructions in output

$$\mathbf{x}_p \in \mathbb{R}^{N \times (P^2 C)}$$

$$N = HW/P^2$$



Then one question raises  
Inductive Bias & Scale



# The Bitter Lesson



Richard S. Sutton

 FOLLOW

Keen, Amii, and University of Alberta  
Verified email at richsutton.com - [Homepage](#)

[artificial intelligence](#) [reinforcement learning](#) [machine learning](#) [cognitive science](#)  
[computer science](#)

| TITLE                                                                                                                                                                                                 | CITED BY | YEAR |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|------|
| <a href="#">Reinforcement learning: An introduction</a><br>RS Sutton, AG Barto<br>MIT press 1 (1), 9-11                                                                                               | 89402    | 1998 |
| <a href="#">Policy gradient methods for reinforcement learning with function approximation</a><br>RS Sutton, D McAllester, S Singh, Y Mansour<br>Advances in neural information processing systems 12 | 10386    | 1999 |
| <a href="#">Learning to predict by the methods of temporal differences</a><br>RS Sutton<br>Machine learning 3 (1), 9-44                                                                               | 8878     | 1988 |

“... general methods that leverage computation are ultimately the most effective, and by a large margin ... ”

Power of Search and Learning



## Experiment (Observation) Model

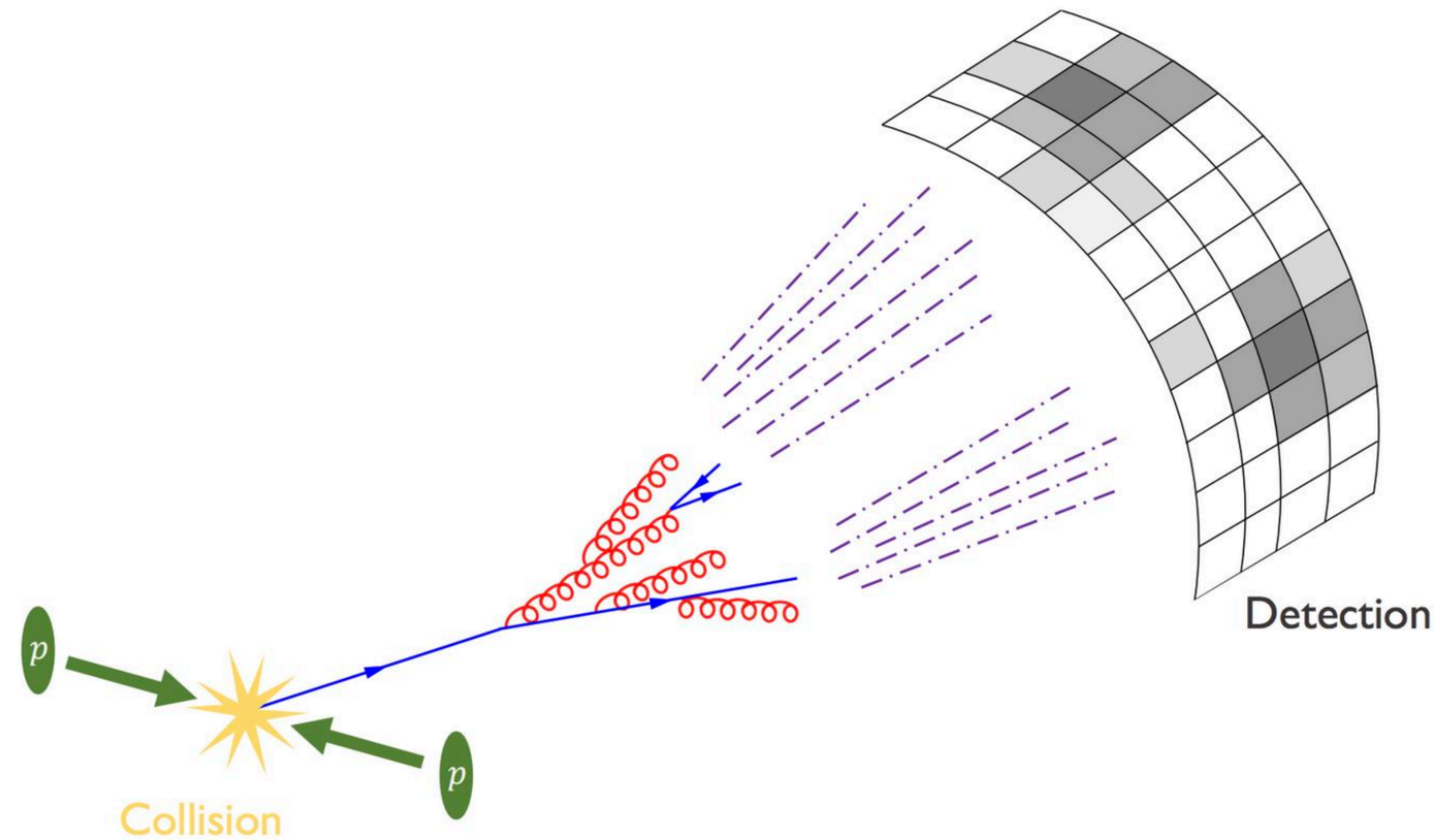
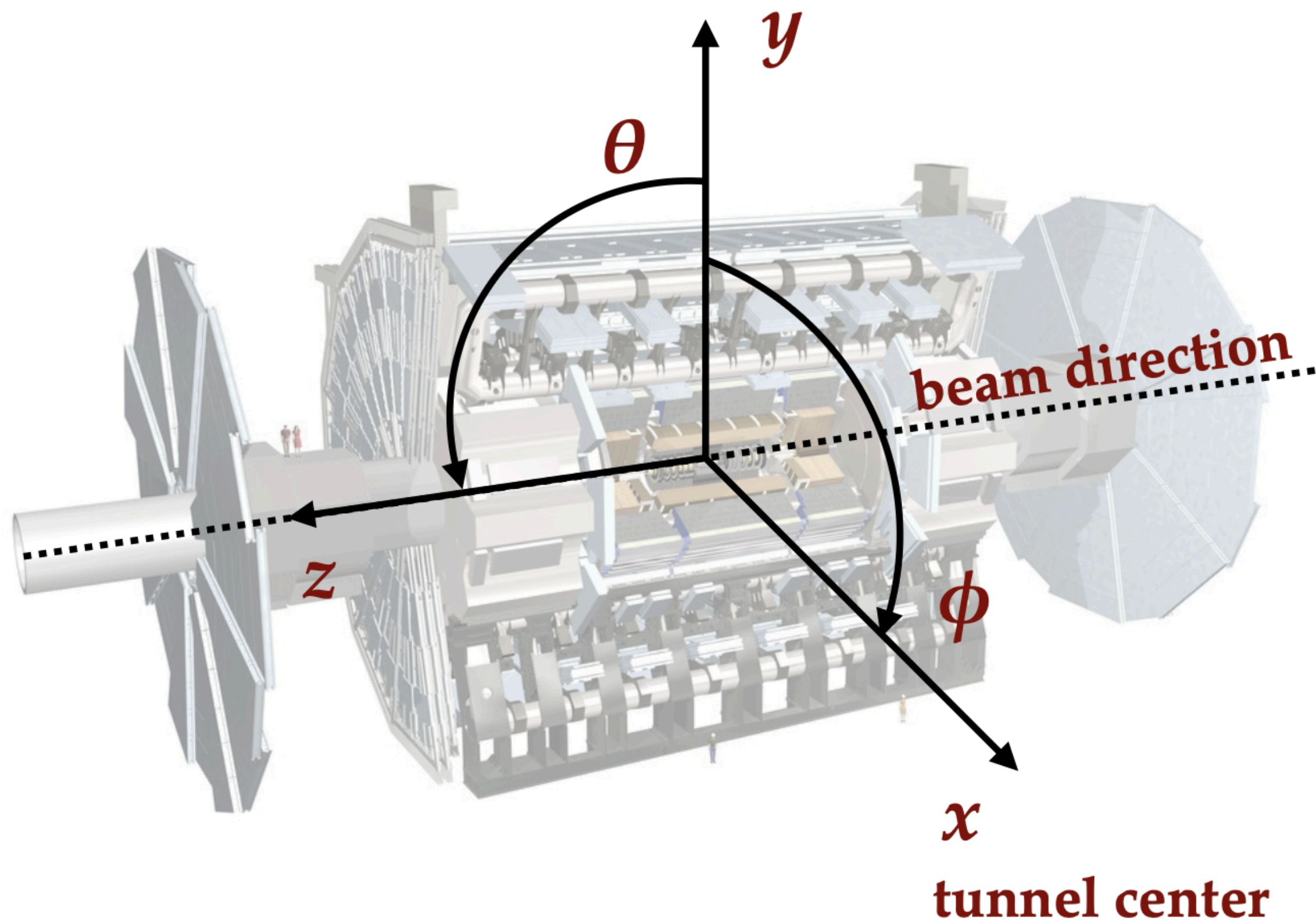
Very initial step

Try to understand ( in natural representation) what is happening there!



## Experiment (Observation) Model

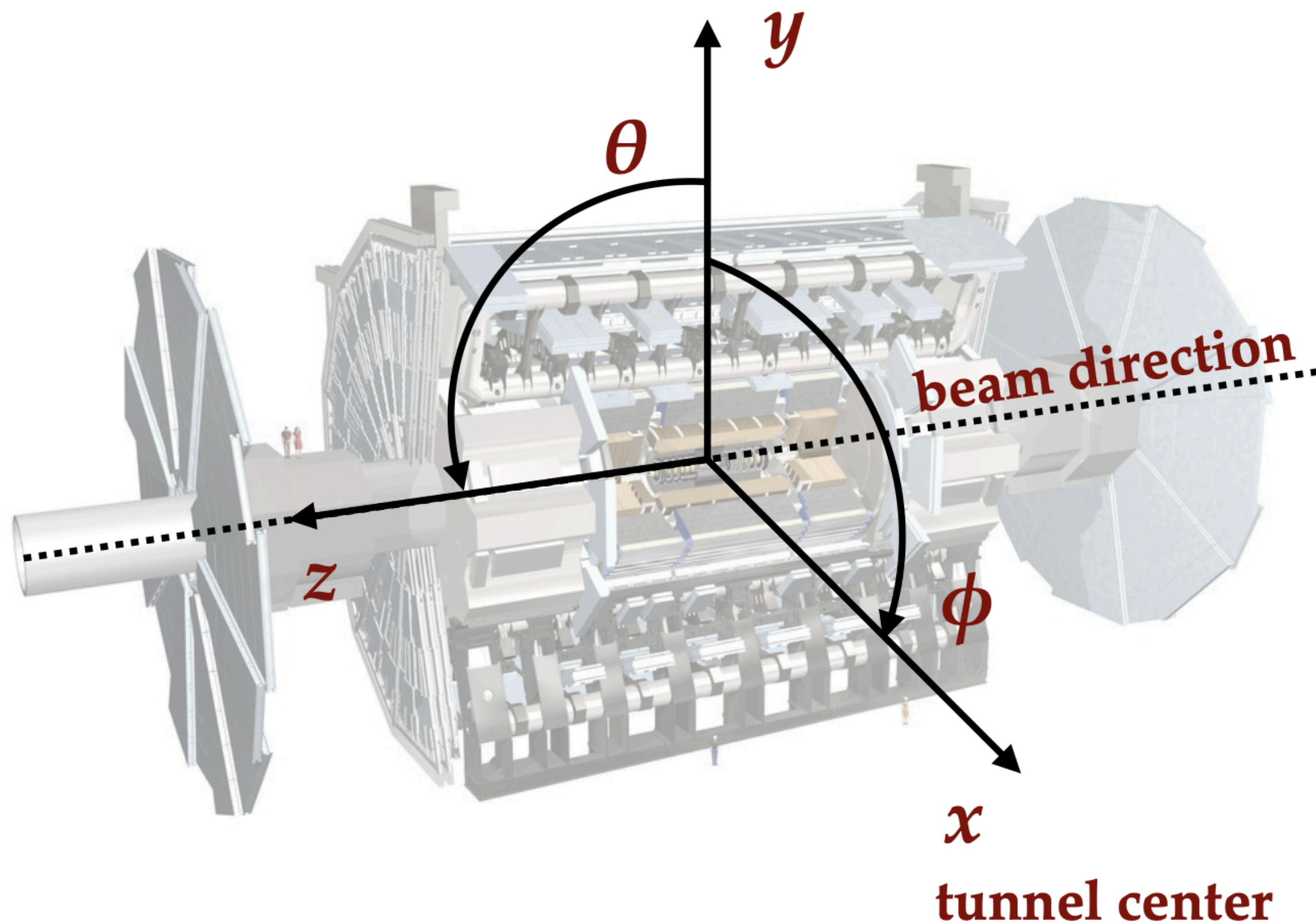
pp collision data from the ATLAS experiment at the Large Hadron Collider (LHC)





## Experiment (Observation) Model

pp collision data from the ATLAS experiment at the Large Hadron Collider (LHC)

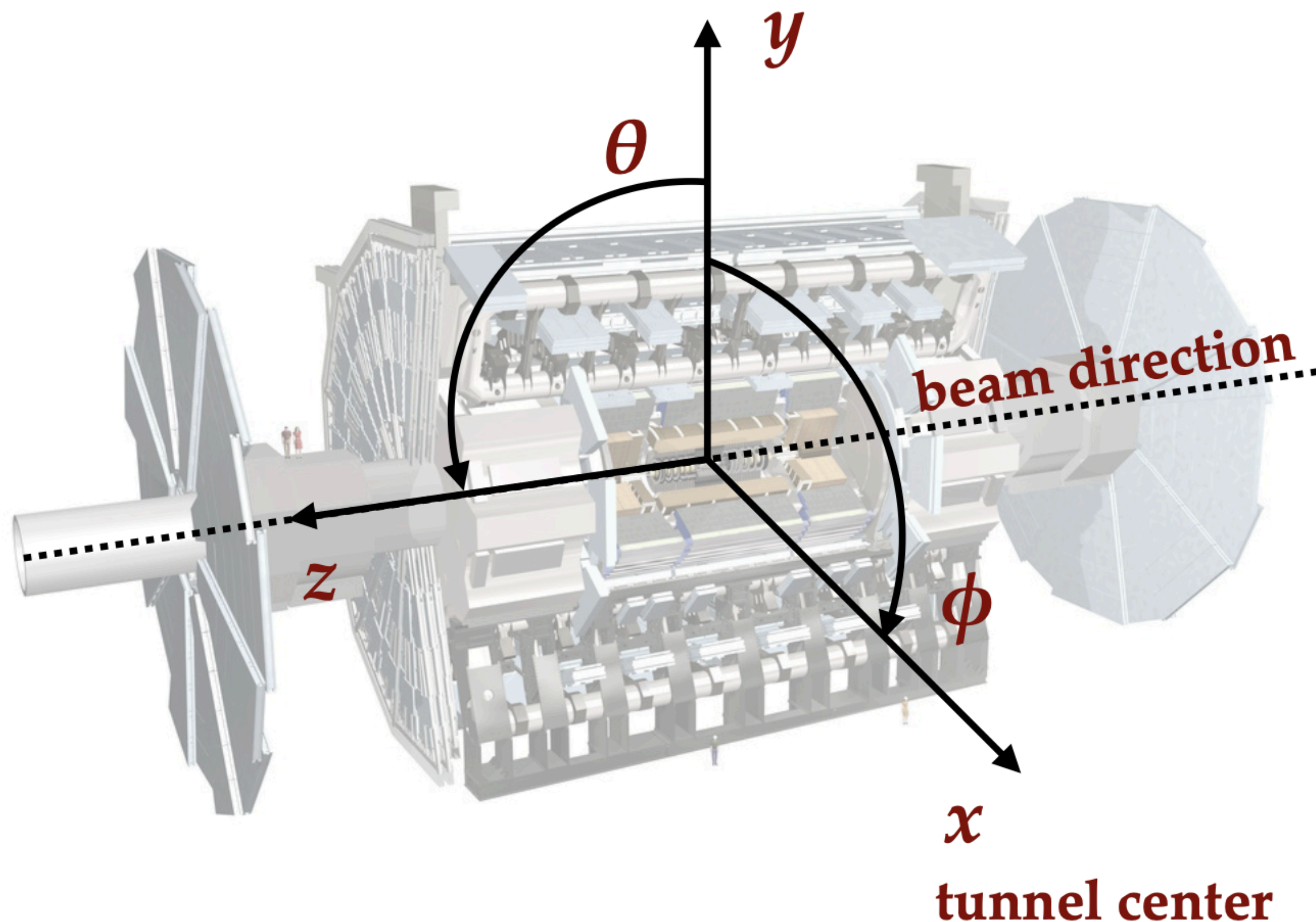


Pseudorapidity

$$\eta = -\log \tan \left( \frac{\theta}{2} \right)$$

## Experiment (Observation) Model

pp collision data from the ATLAS experiment at the Large Hadron Collider (LHC)



Pseudorapidity

$$\eta = -\log \tan \left( \frac{\theta}{2} \right)$$

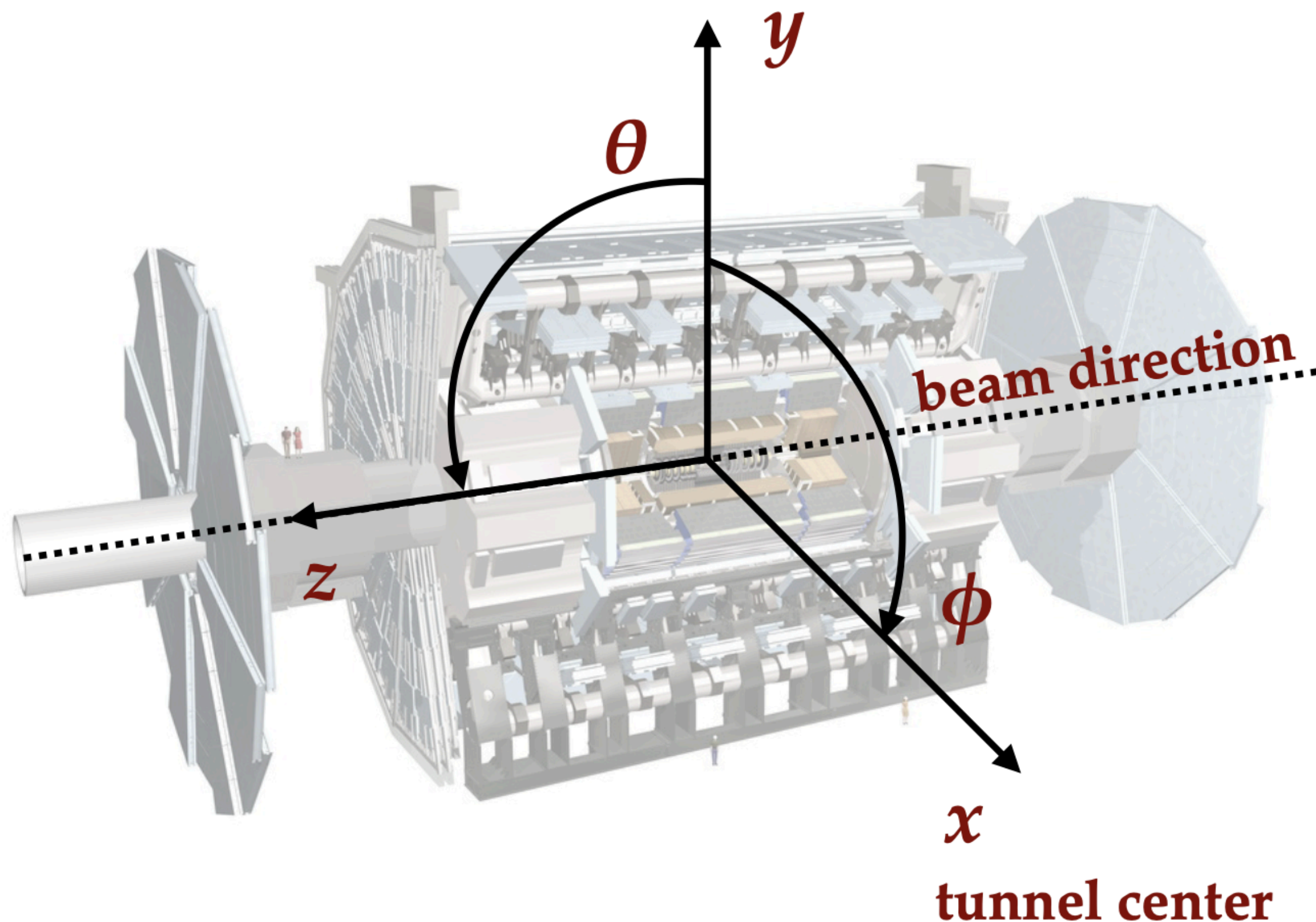
Momentum transverse

$$p_T \equiv \sqrt{p_x^2 + p_y^2}$$



## Experiment (Observation) Model

pp collision data from the ATLAS experiment at the Large Hadron Collider (LHC)



Pseudorapidity

$$\eta = -\log \tan \left( \frac{\theta}{2} \right)$$

Momentum transverse

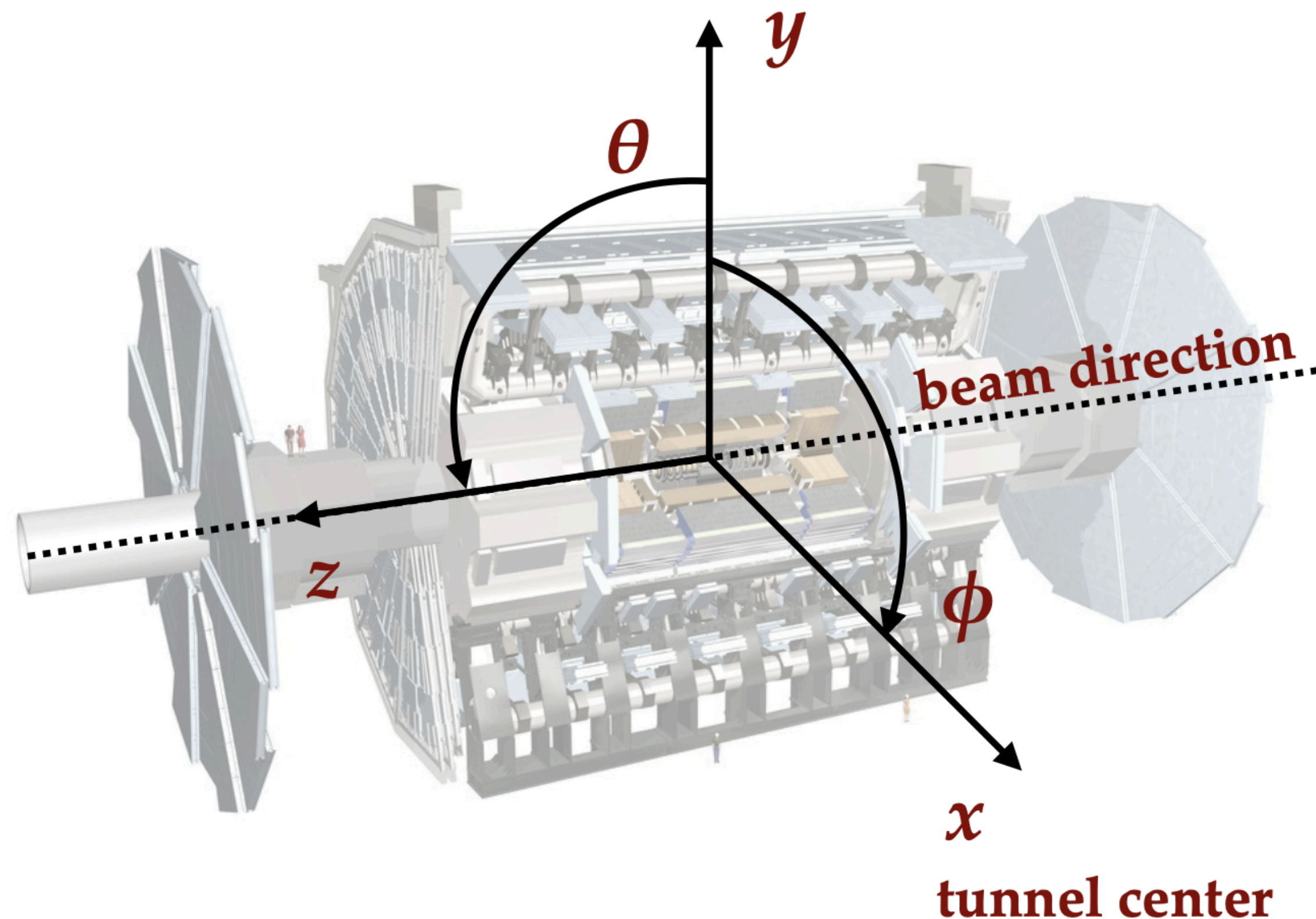
$$p_T \equiv \sqrt{p_x^2 + p_y^2}$$

Particle feature vector

$$p = (\phi, \eta, p_T)$$

## Experiment (Observation) Model

pp collision data from the ATLAS experiment at the Large Hadron Collider (LHC)



### Quiz :

100 Billion of protons (a Bunch) cross each other with a rate of 40 Million per second

Average number of interaction per bunch is  $\sim 50$

Total cross section  $\sigma \sim 80$  mb pileup, ( $1 \text{ b} = 10^{-24} \text{ cm}^2$ )

A) Find Luminosity in per  $\text{cm}^2$  per second

Proton Hard scattering cross section  $\sigma \sim 1$  mb (jets),

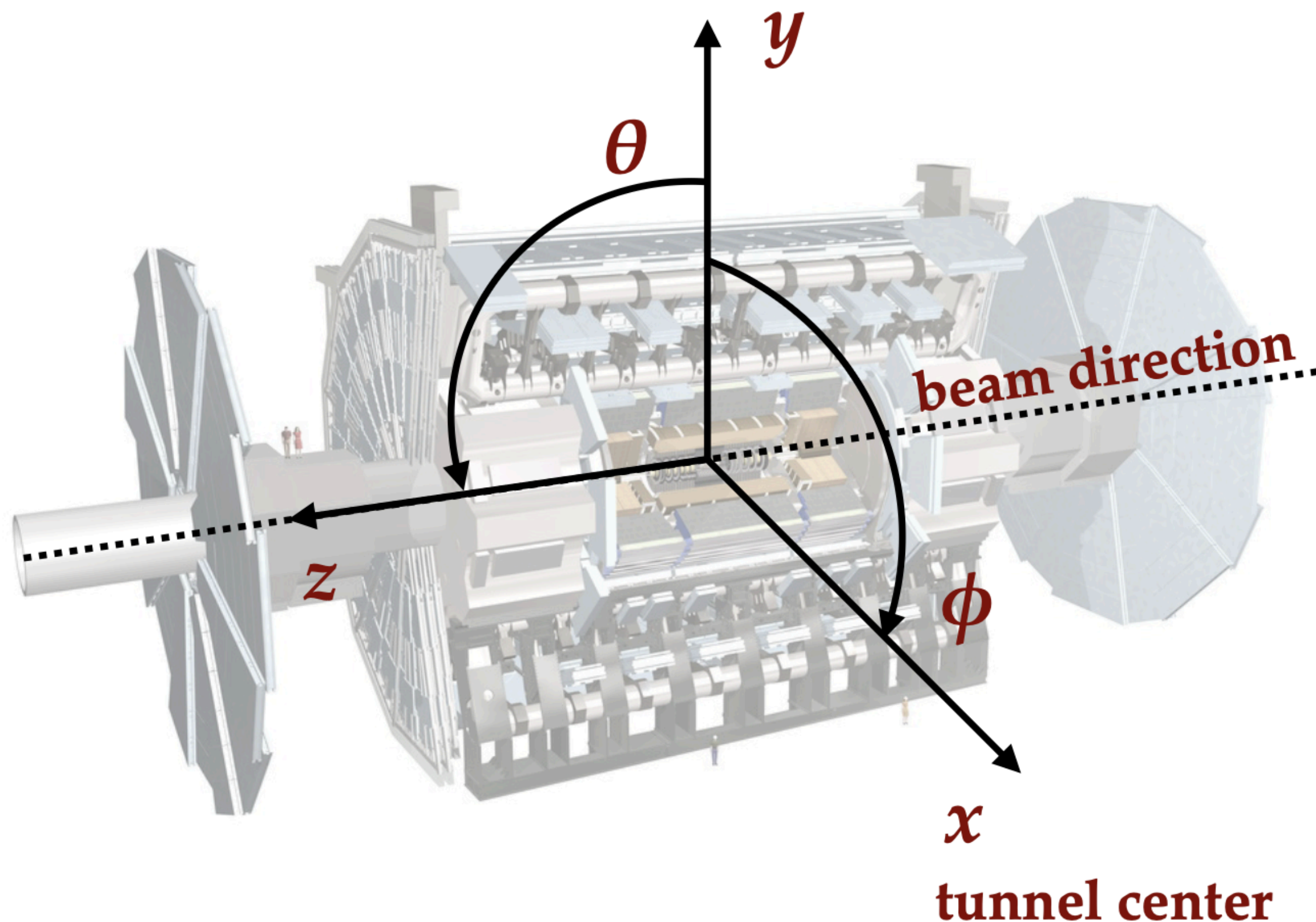
Rare scattering cross section  $\sigma \sim 50$  pb (higgs), one pico is  $10^{-12}$

B) Find number of Pileup, Jets and Higgs per second ?



## Experiment (Observation) Model

pp collision data from the ATLAS experiment at the Large Hadron Collider (LHC)



### Quiz :

100 Billion of protons (a Bunch) cross each other with a rate of 40 Million per second

Average number of interaction per bunch is  $\sim 50$

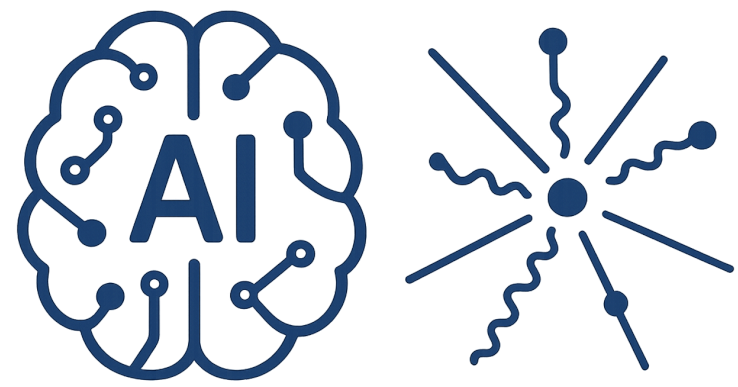
Total cross section  $\sigma \sim 80 \text{ mb}$  pileup, ( $1 \text{ b} = 10^{-24} \text{ cm}^2$ )

$$\text{A) } L = \frac{50 \times 40 \times 10^6 \text{ s}^{-1}}{80 \times 10^{-3} \times 10^{-24} \text{ cm}^2} = 2.5 \times 10^{34} \text{ cm}^{-2} \text{ s}^{-1}$$

Proton Hard scattering cross section  $\sigma \sim 1 \text{ mb}$  (jets),

Rare scattering cross section  $\sigma \sim 50 \text{ pb}$  (higgs), one pico is  $10^{-12}$

$$\text{B) Pileup } \sim 20 \times 10^8 \text{ s}^{-1}, \text{ Jets } \sim 2.5 \times 10^7 \text{ s}^{-1}, \text{ Higgs } \sim 1 \text{ s}^{-1}$$



AI-Driven HEP 19

## Experiment (Observation) Model

For a 40 Million bunch crossed per second

Order of Pileup events 1 Billion per second

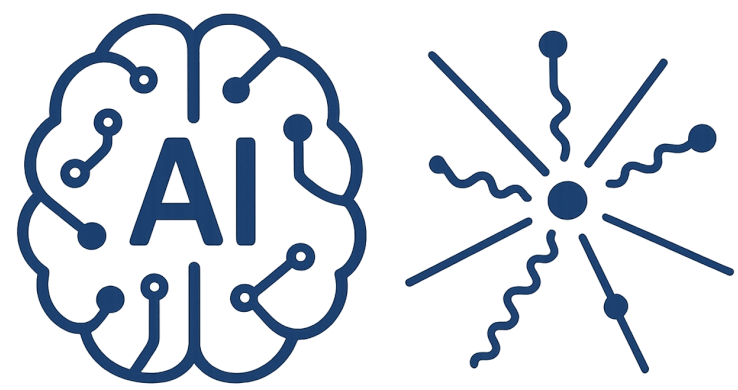
Each event contains  $\sim 50$  particles

Order of Jet events 10 Million per second

Each event contains  $\sim 100$  particles

Do we store all this data ?





## Experiment (Observation) Model

### Trigger

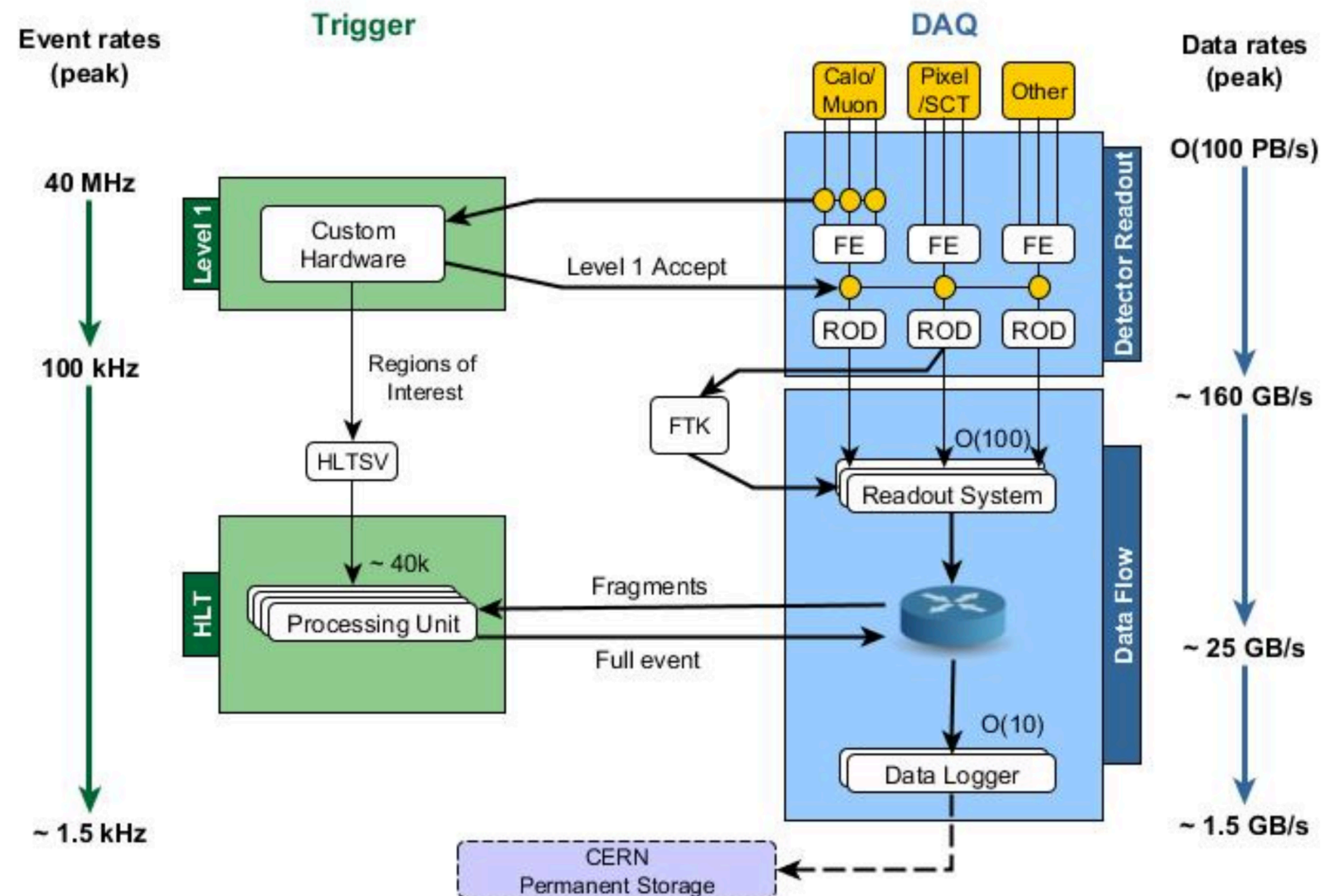
Every 25 ns :  $\sim 50$  collisions

L1 decision ( $2.5 \mu\text{s}$ ) : accept or reject the whole 25 ns frame

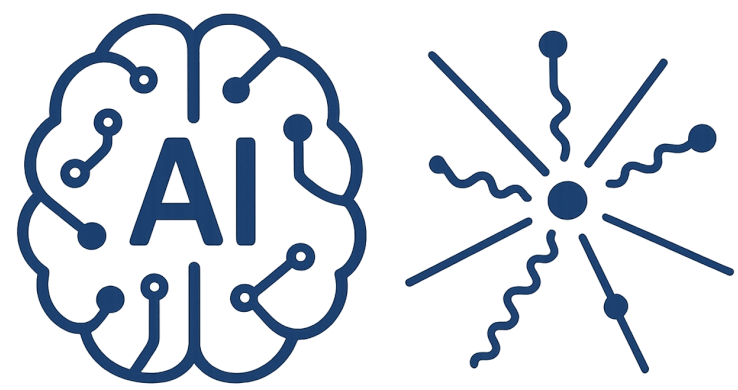
L1 accepts  $\sim 1$  in 400 bunch crossings

HLT (few ms) further accepts  $\sim 1$  in 100 of those

Final stored rate:  $\sim 1$  kHz







## Experiment (Observation) Model

### Trigger

Every 25 ns :  $\sim 50$  collisions

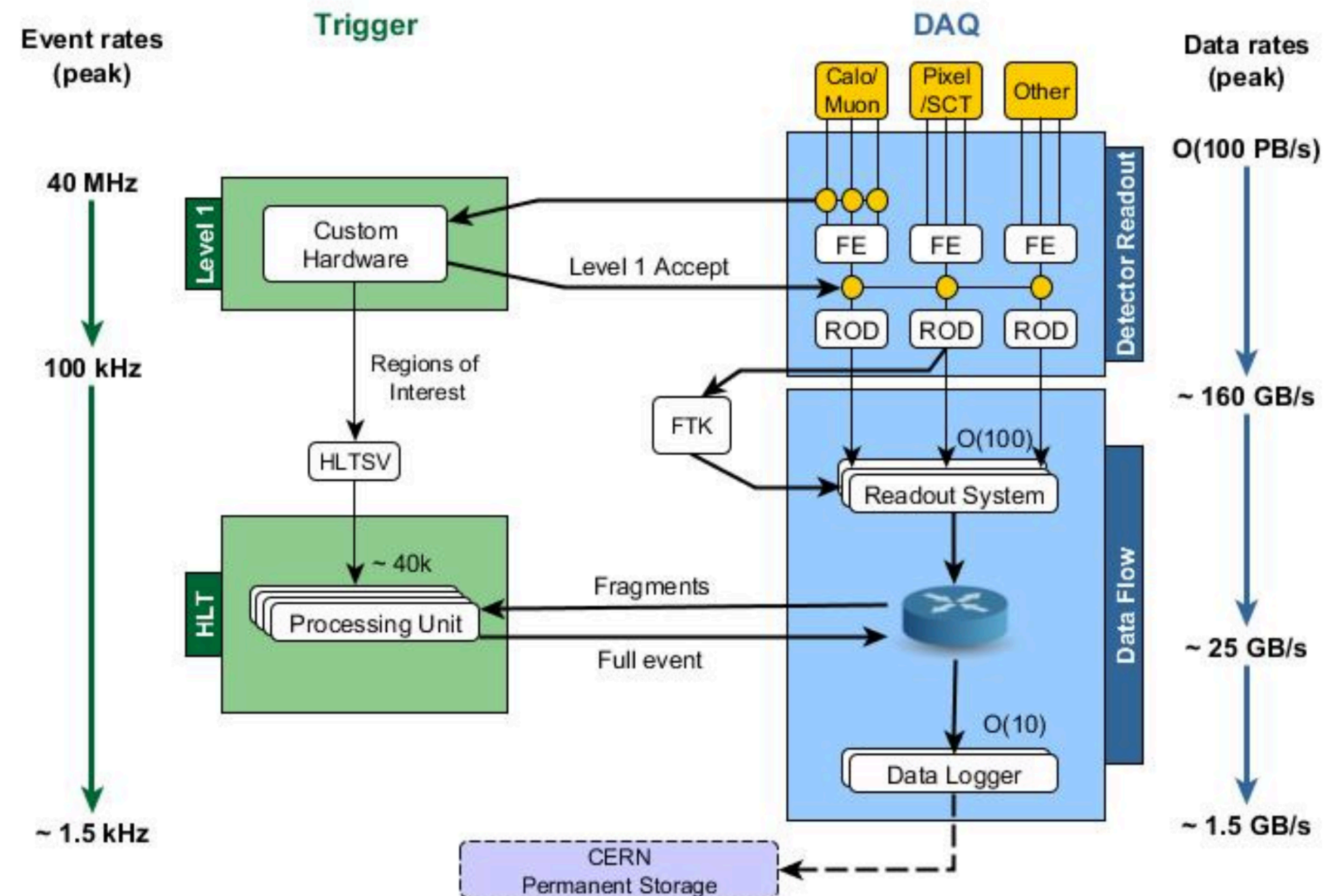
L1 decision ( $2.5 \mu\text{s}$ ) : accept or reject the whole 25 ns frame

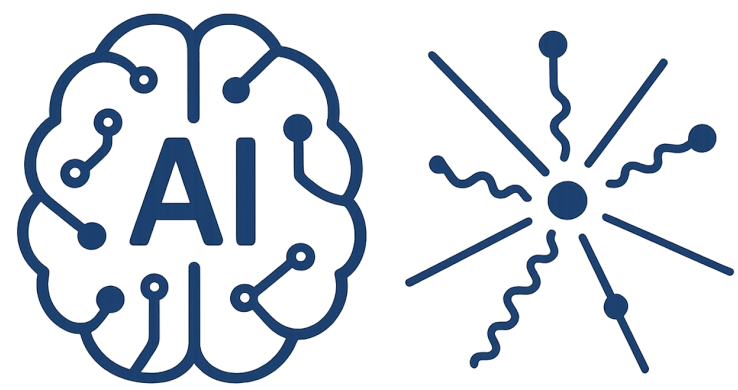
L1 accepts  $\sim 1$  in 400 bunch crossings

HLT (few ms) further accepts  $\sim 1$  in 100 of those

Final stored rate:  $\sim 1$  kHz

Quiz : Order of memory for L1 Trigger ?





## Experiment (Observation) Model

### Trigger

Every 25 ns :  $\sim 50$  collisions

L1 decision ( $2.5 \mu\text{s}$ ) : accept or reject the whole 25 ns frame

L1 accepts  $\sim 1$  in 400 bunch crossings

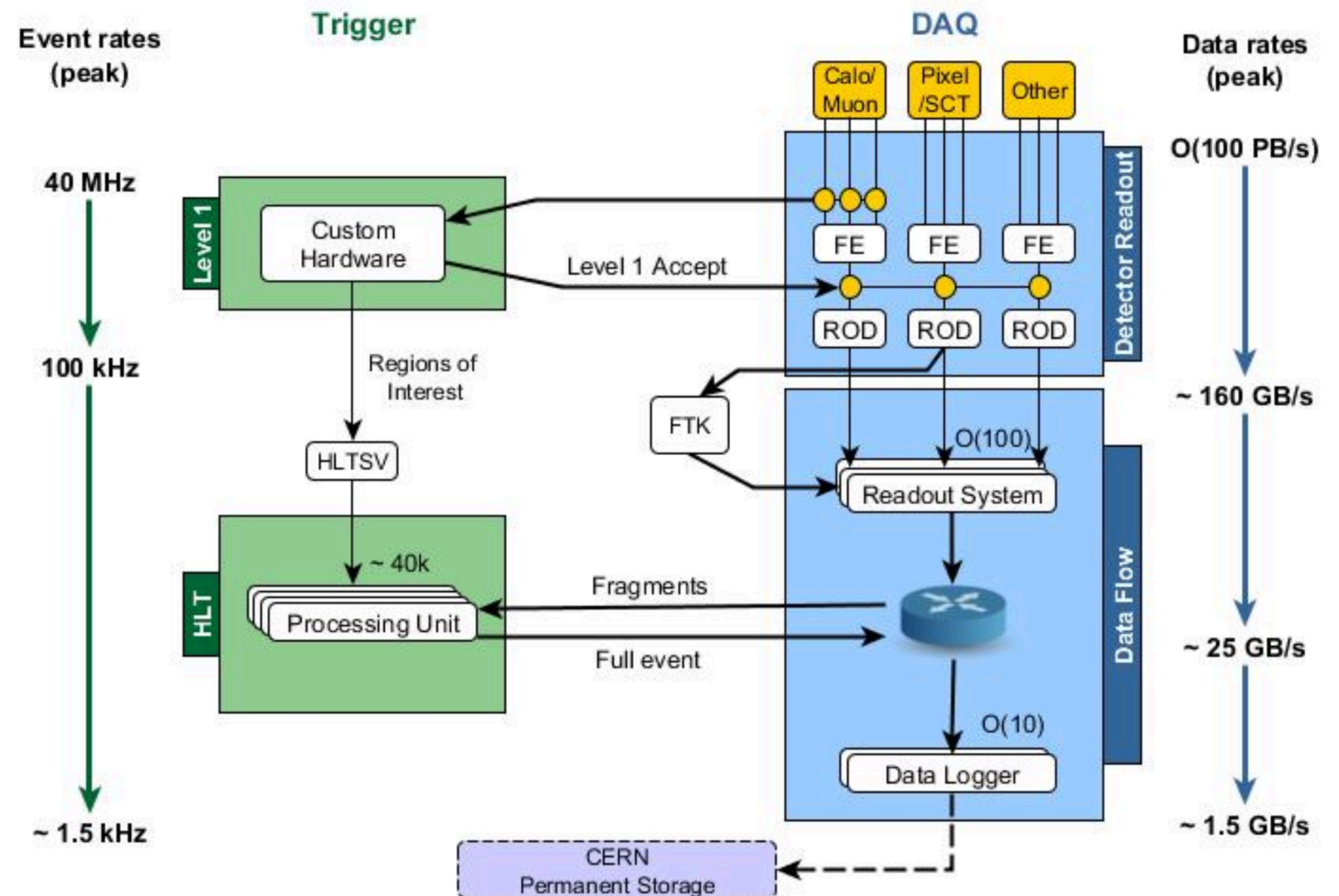
HLT (few ms) further accepts  $\sim 1$  in 100 of those

Final stored rate:  $\sim 1$  kHz

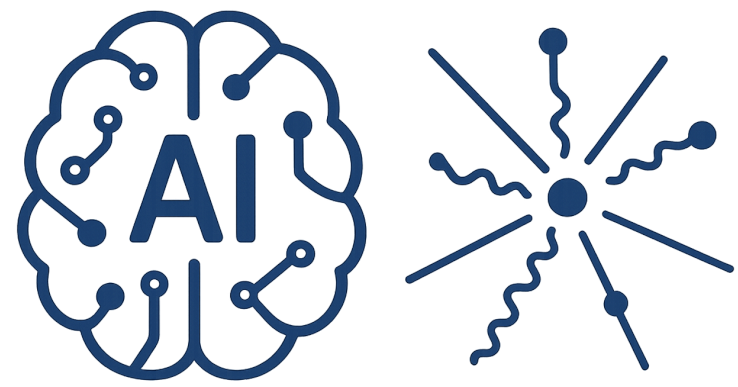
**Quiz : Order of memory for L1 Trigger ?**

$$160 \text{ GB} / 100,000 = 1.6 \text{ MB}$$

$$1.6 \text{ MB} \times 2.5 / (25 \times 10^3) = 160 \text{ MB}$$







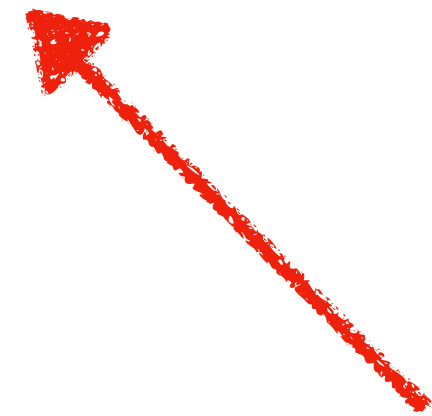
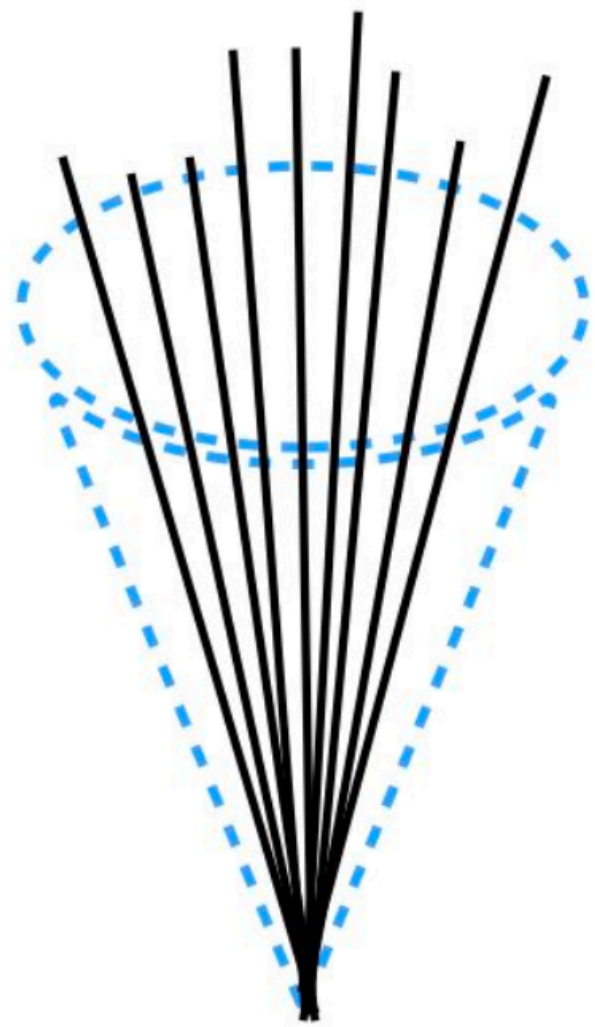
## Jet Tagging

We want to label each Jet : classification task

Each jet contain N particle each has K features

Point cloud ☁ ( N, K )

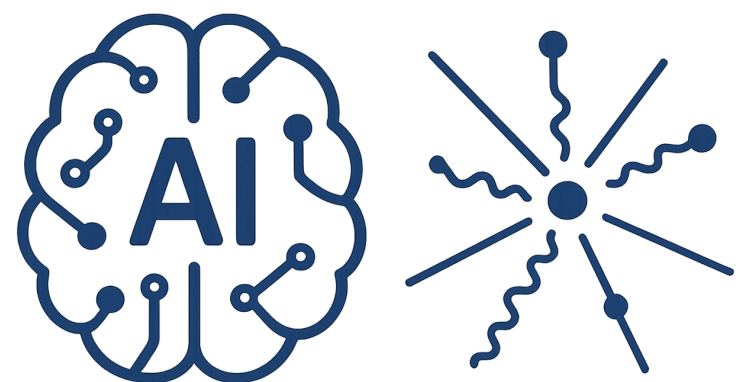
$$p = (\phi, \eta, \log p_T, \log E)$$



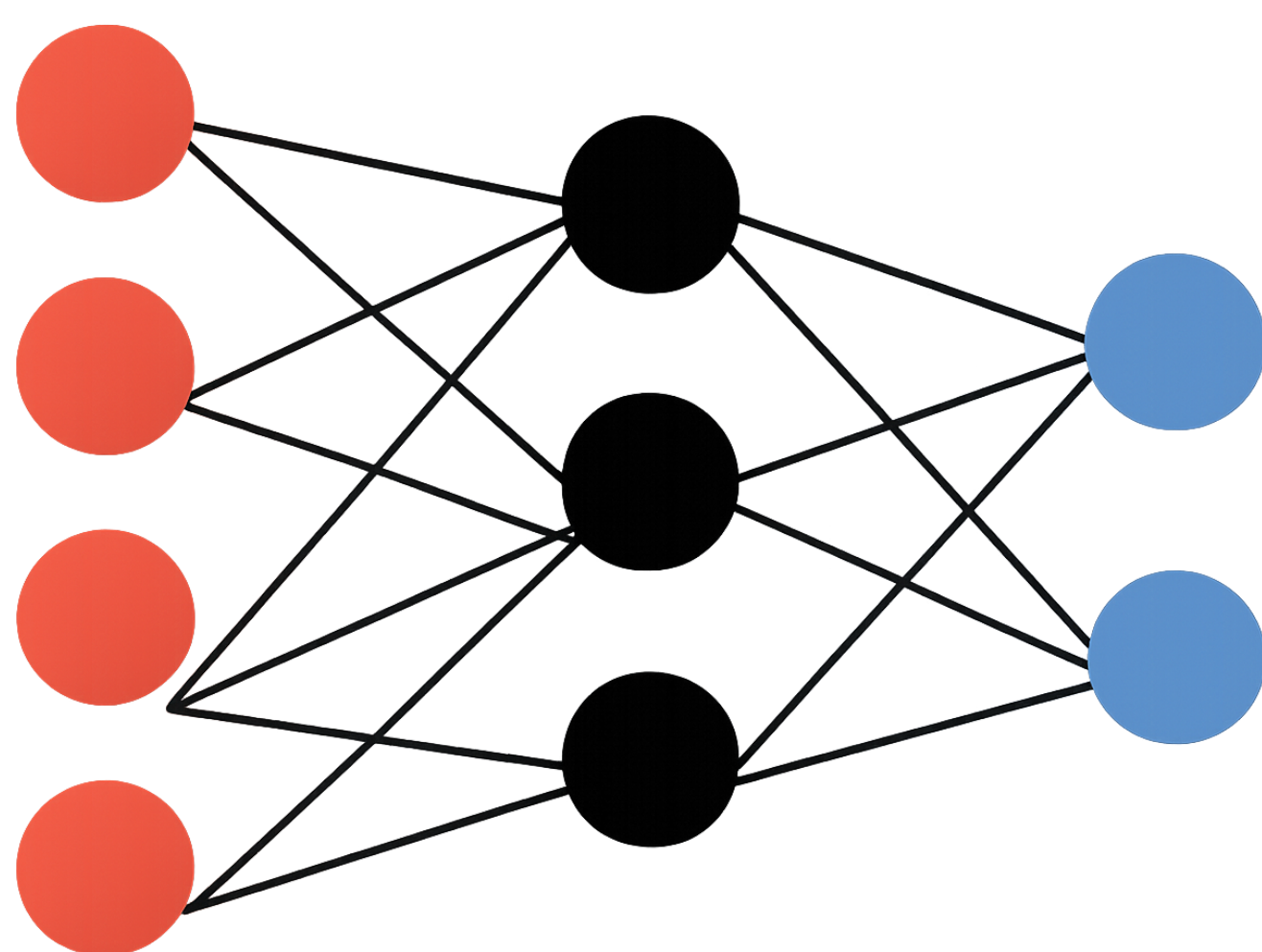
Angle with respect to the Jet axis

Model needs to learn the relative positions of particles in a Jet





## FCDN



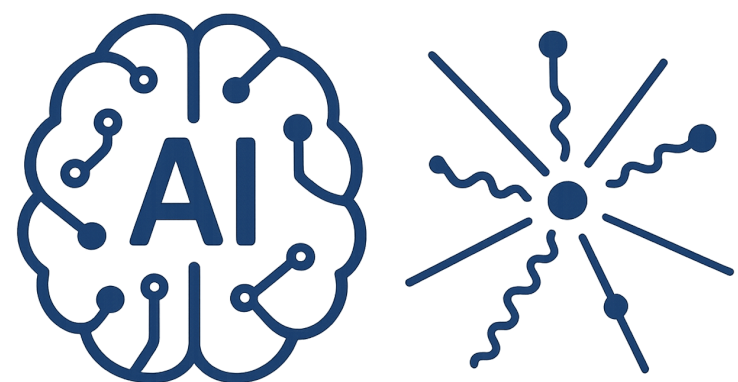
$$X = \begin{bmatrix} X_1 \\ X_2 \\ \vdots \\ X_N \end{bmatrix}$$

$$X_i = [\phi_i, \eta_i, pT_i, E_i]$$

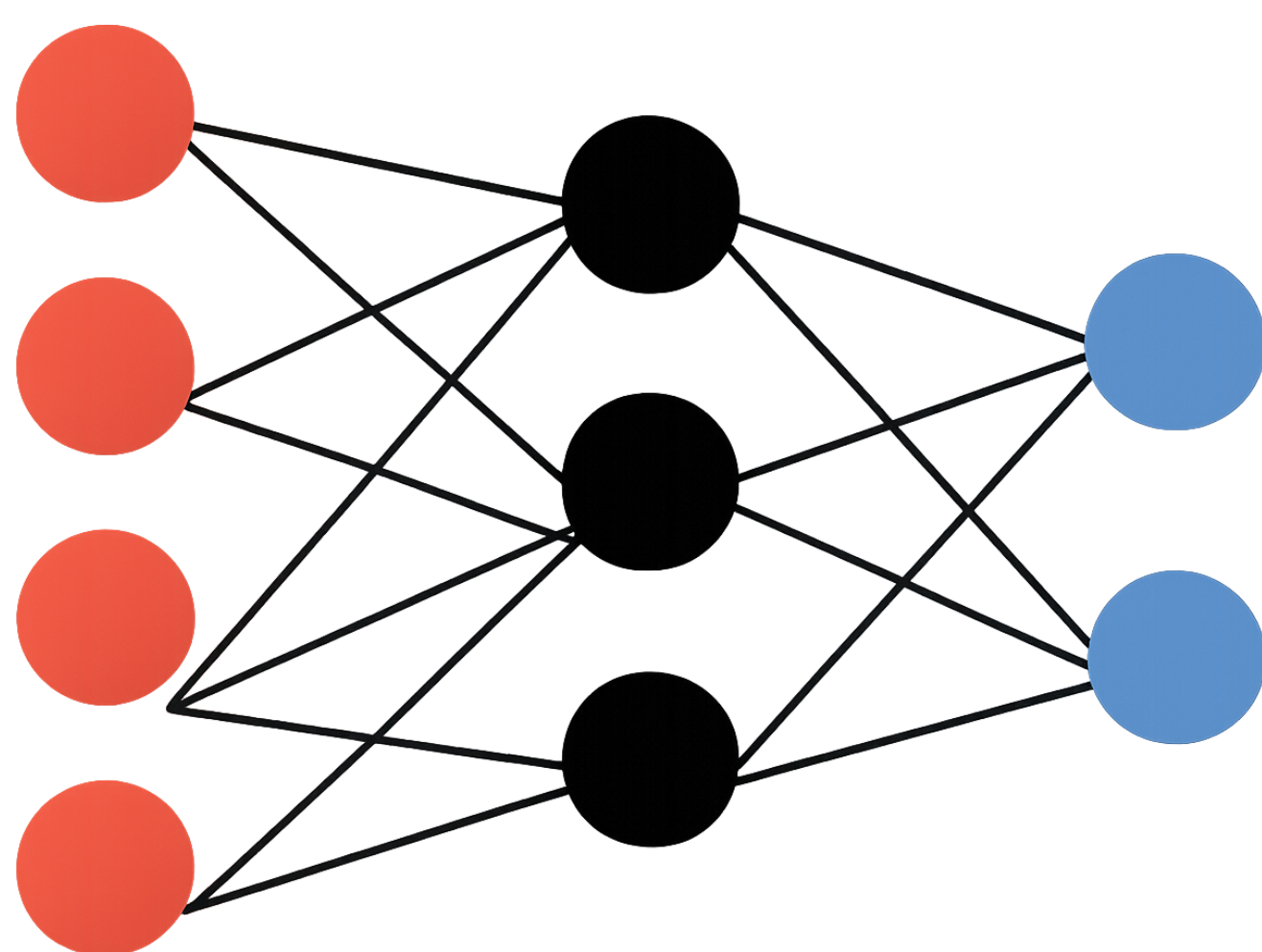
$$W = \begin{bmatrix} \vec{w}_1 \\ \vec{w}_2 \\ \vec{w}_3 \end{bmatrix}$$

$$XW^T = \begin{bmatrix} \vec{K}_1 \\ \vec{K}_2 \\ \vdots \\ \vec{K}_N \end{bmatrix}$$

Compute  $\vec{K}_i$  ?



## FCDN



$$X = \begin{bmatrix} X_1 \\ X_2 \\ \vdots \\ X_N \end{bmatrix}$$

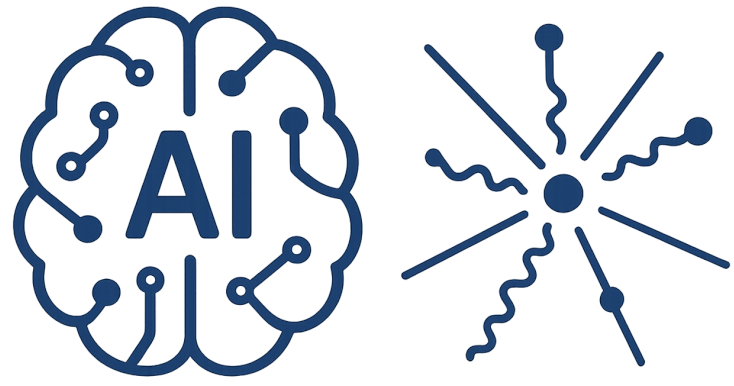
$$X_i = [\phi_i, \eta_i, pT_i, E_i]$$

$$W = \begin{bmatrix} \vec{w}_1 \\ \vec{w}_2 \\ \vec{w}_3 \end{bmatrix}$$

$$XW^T = \begin{bmatrix} \vec{K}_1 \\ \vec{K}_2 \\ \vdots \\ \vec{K}_N \end{bmatrix}$$

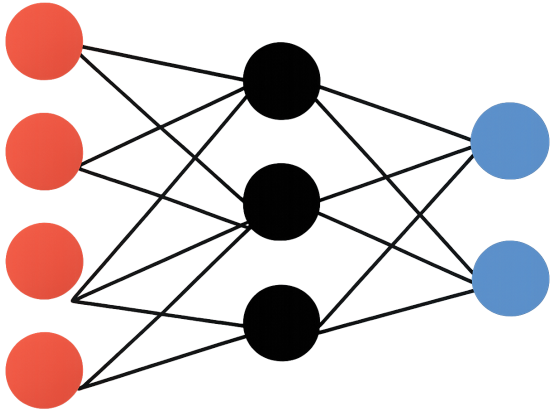
Compute  $\vec{K}_i$  ?

$$\vec{K}_i = [X_i \cdot \vec{w}_1, X_i \cdot \vec{w}_2, X_i \cdot \vec{w}_3]$$



# AI-Driven HEP 19

## DeepSets



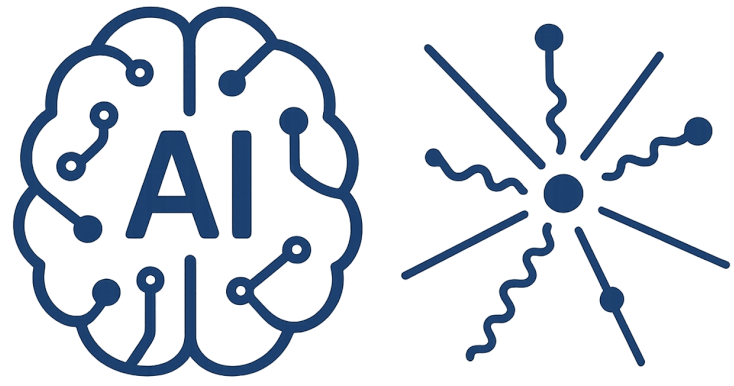
```
class SimpleDeepSets(nn.Module):
    def __init__(self, input_dim, config, num_classes=2):
        super().__init__()
        self.input_layer = nn.Linear(input_dim, config["hidden_dim"])

        layers = []
        for _ in range(config["num_layers"]):
            layers.append(nn.Linear(config["hidden_dim"], config["hidden_dim"]))
            layers.append(config["activation"])
        self.hidden_layers = nn.ModuleList(layers)

        self.output_layer = nn.Linear(config["hidden_dim"], num_classes)

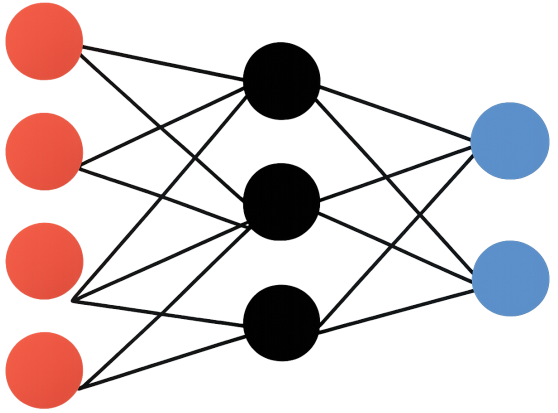
    def forward(self, inputs):
        zero_pad_mask = (inputs[:, :, 2] != 0).unsqueeze(-1).float()
        x = self.input_layer(inputs) * zero_pad_mask
        for layer in self.hidden_layers:
            x = layer(x) * zero_pad_mask
        x = x.mean(1) # aggregate over particles
        return self.output_layer(x)
```





# AI-Driven HEP 19

## DeepSets



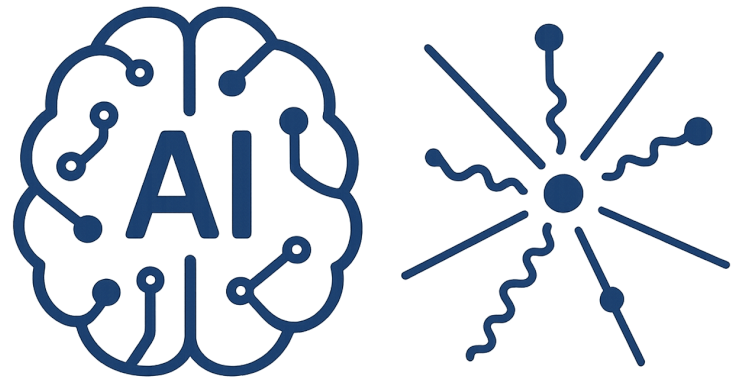
- Batch size (B): 2
- Max particles (N): 4
- Features (K): 5
- Feature at index 2 (the 3rd feature) is used as sentinel: 0 = padded, non-zero = real

Sample 1 (3 real particles, 1 padding):

```
[[[1.2, 3.4, 5.6, 7.8, 9.0], # particle 1 - real (5.6 ≠ 0)
  [2.3, 4.5, 6.7, 8.9, 0.1], # particle 2 - real (6.7 ≠ 0)
  [3.4, 5.6, 7.8, 9.0, 1.2], # particle 3 - real (7.8 ≠ 0)
  [0.0, 0.0, 0.0, 0.0, 0.0]] # particle 4 - padded (0.0 = 0)
```

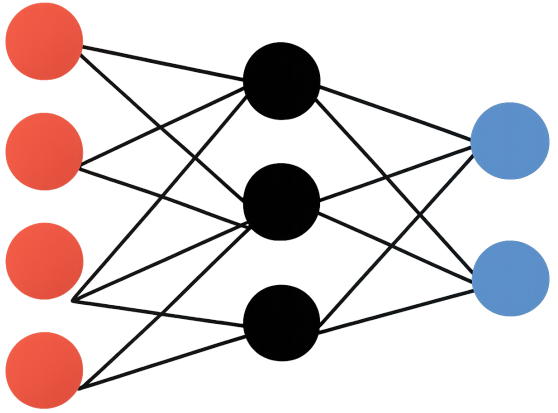
Sample 2 (2 real particles, 2 padding):

```
[[[4.5, 6.7, 8.9, 0.1, 2.3], # particle 1 - real (8.9 ≠ 0)
  [5.6, 7.8, 9.0, 1.2, 3.4], # particle 2 - real (9.0 ≠ 0)
  [0.0, 0.0, 0.0, 0.0, 0.0], # particle 3 - padded (0.0 = 0)
  [0.0, 0.0, 0.0, 0.0, 0.0]] # particle 4 - padded (0.0 = 0)
```



# AI-Driven HEP 19

## DeepSets



**# Extract feature at index 2 (3rd feature)**

```
inputs[:, :, 2] =
```

```
[  
    # Sample 1  
    [5.6, 6.7, 7.8, 0.0],  
  
    # Sample 2  
    [8.9, 9.0, 0.0, 0.0]  
]
```

**# Check which are non-zero**

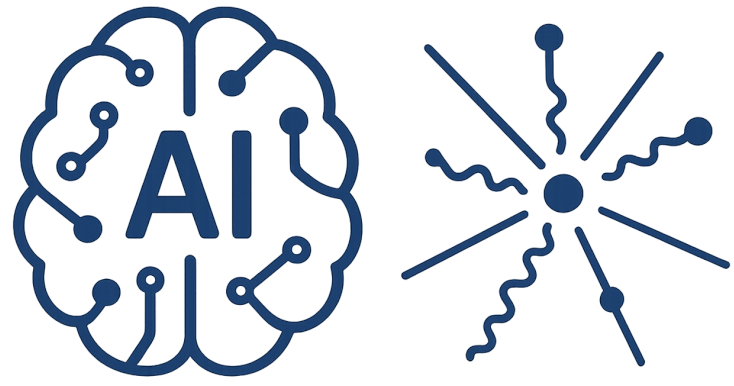
```
(inputs[:, :, 2] != 0) =  
[  
    [True, True, True, False], # Sample 1  
    [True, True, False, False] # Sample 2  
]
```

**# Unsqueeze to add dimension: (B, N, 1)**

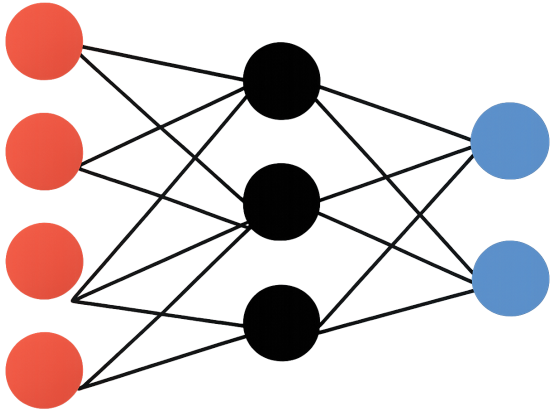
```
zero_pad_mask =  
[  
    # Sample 1  
    [[True], [True], [True], [False]],  
  
    # Sample 2  
    [[True], [True], [False], [False]]  
]
```

**# Convert to float**

```
zero_pad_mask =  
[  
    # Sample 1  
    [[1.0], [1.0], [1.0], [0.0]],  
  
    # Sample 2  
    [[1.0], [1.0], [0.0], [0.0]]  
]
```



## DeepSets



Sample 1 after input\_layer:

```
[[[1.1, 2.2, 3.3], # particle 1  
  [2.2, 3.3, 4.4], # particle 2  
  [3.3, 4.4, 5.5], # particle 3  
  [0.1, 0.2, 0.3]], # particle 4 (from padding!]
```

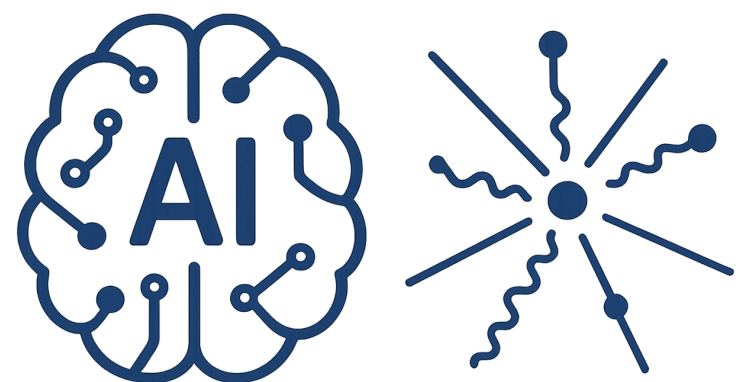
Sample 2 after input\_layer:

```
[[[4.4, 5.5, 6.6], # particle 1  
  [5.5, 6.6, 7.7], # particle 2  
  [0.1, 0.2, 0.3], # particle 3 (from padding!  
  [0.1, 0.2, 0.3]] # particle 4 (from padding!]
```

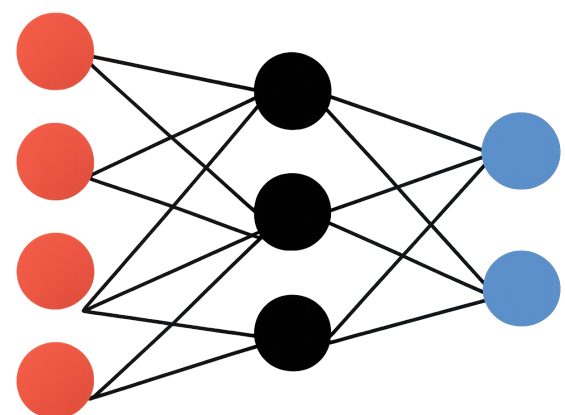
```
[[[1.1*1.0, 2.2*1.0, 3.3*1.0], = [1.1, 2.2, 3.3]  
  [2.2*1.0, 3.3*1.0, 4.4*1.0], = [2.2, 3.3, 4.4]  
  [3.3*1.0, 4.4*1.0, 5.5*1.0], = [3.3, 4.4, 5.5]  
  [0.1*0.0, 0.2*0.0, 0.3*0.0]]] = [0.0, 0.0, 0.0] ← Zeroed out!
```

```
[[[4.4*1.0, 5.5*1.0, 6.6*1.0], = [4.4, 5.5, 6.6]  
  [5.5*1.0, 6.6*1.0, 7.7*1.0], = [5.5, 6.6, 7.7]  
  [0.1*0.0, 0.2*0.0, 0.3*0.0], = [0.0, 0.0, 0.0] ← Zeroed out!  
  [0.1*0.0, 0.2*0.0, 0.3*0.0]]] = [0.0, 0.0, 0.0] ← Zeroed out!
```





## DeepSets

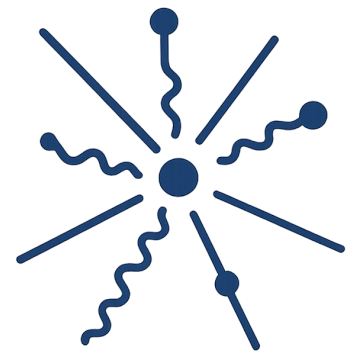
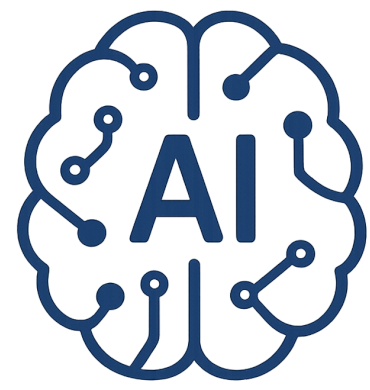


Average of 3 particles (not 4!):

$$\begin{aligned} & ( [1.1, 2.2, 3.3] + [2.2, 3.3, 4.4] + [3.3, 4.4, 5.5] ) / 3 \\ &= [6.6, 9.9, 13.2] / 3 \\ &= [2.2, 3.3, 4.4] \end{aligned}$$

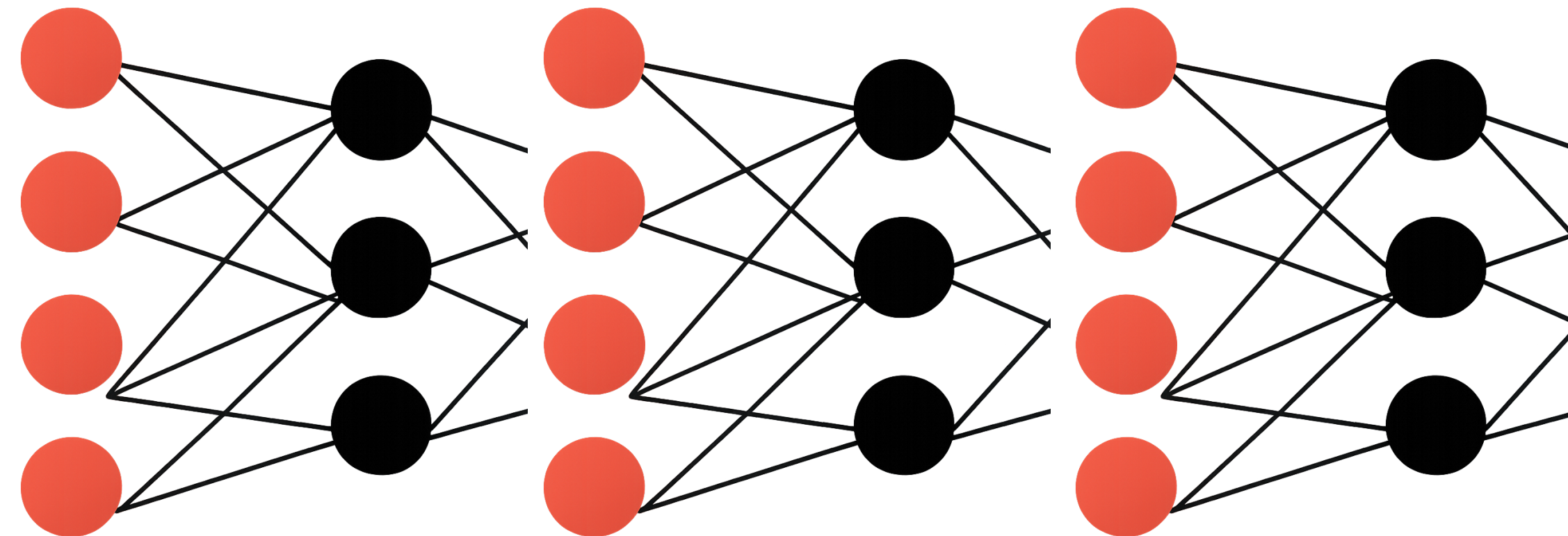
Average of 2 particles (not 4!):

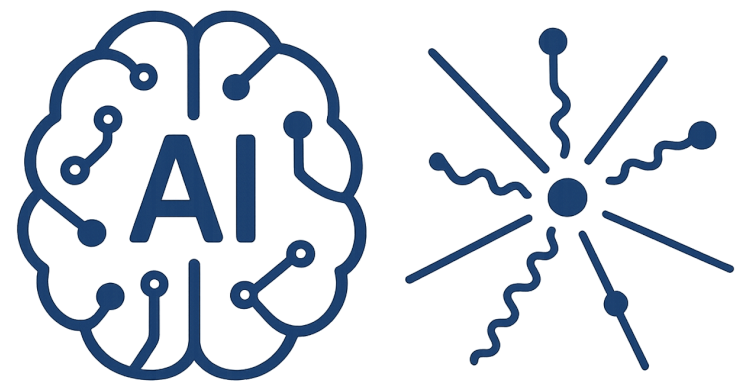
$$\begin{aligned} & ( [4.4, 5.5, 6.6] + [5.5, 6.6, 7.7] ) / 2 \\ &= [9.9, 12.1, 14.3] / 2 \\ &= [4.95, 6.05, 7.15] \end{aligned}$$



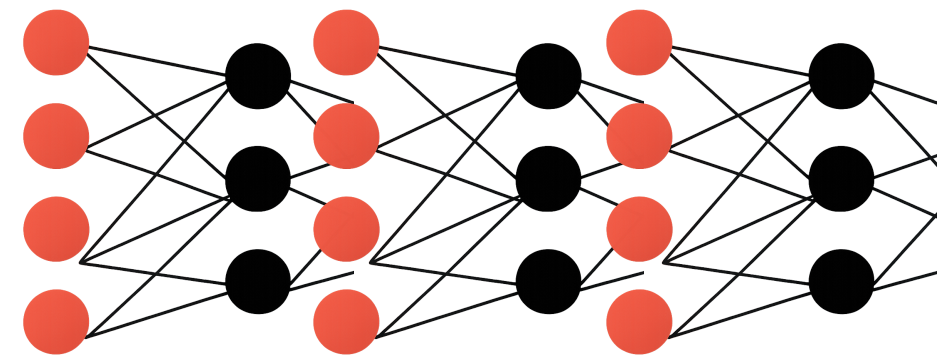
## DeepTransformerSets

- 1) Structure of inter-particle in a Jet
- 2) Unaffected the noise particles





## DeepTransformerSets



$$K = XW_k^T = \begin{bmatrix} \vec{K}_1 \\ \vec{K}_2 \\ \vdots \\ \vec{K}_N \end{bmatrix} \quad \vec{K}_i = [X_i \cdot \vec{W}_{k1}, X_i \cdot \vec{W}_{k2}, X_i \cdot \vec{W}_{k3}]$$

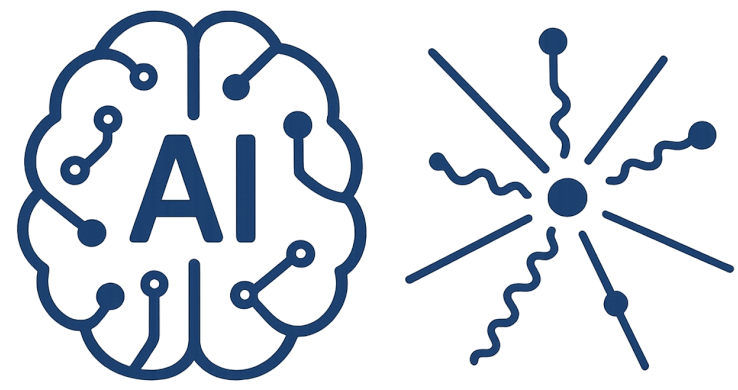
$$X = \begin{bmatrix} X_1 \\ X_2 \\ \vdots \\ X_N \end{bmatrix} \quad W_k = \begin{bmatrix} \vec{w}_{k1} \\ \vec{w}_{k2} \\ \vec{w}_{k3} \end{bmatrix}$$

$$Q = XW_q^T = \begin{bmatrix} \vec{Q}_1 \\ \vec{Q}_2 \\ \vdots \\ \vec{Q}_N \end{bmatrix} \quad \vec{Q}_i = [X_i \cdot \vec{W}_{q1}, X_i \cdot \vec{W}_{q2}, X_i \cdot \vec{W}_{q3}]$$

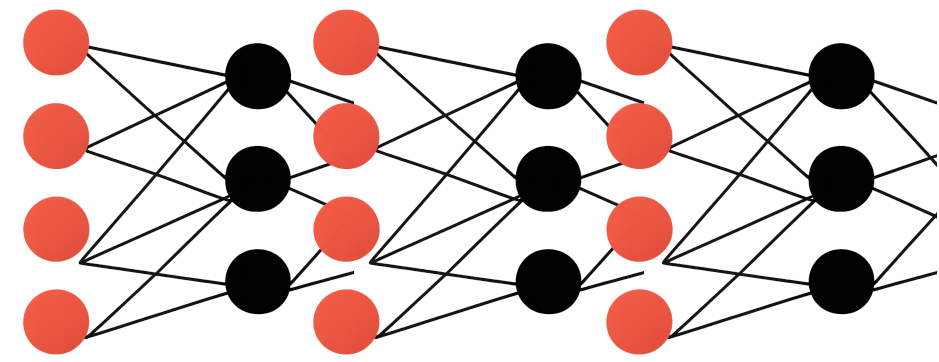
$$X_i = [\phi_i, \eta_i, pT_i, E_i]$$

$$V = XW_v^T = \begin{bmatrix} \vec{V}_1 \\ \vec{V}_2 \\ \vdots \\ \vec{V}_N \end{bmatrix} \quad \vec{V}_i = [X_i \cdot \vec{W}_{v1}, X_i \cdot \vec{W}_{v2}, X_i \cdot \vec{W}_{v3}]$$





## DeepTransformerSets



$$K = XW_k^T = \begin{bmatrix} \vec{K}_1 \\ \vec{K}_2 \\ \vdots \\ \vec{K}_N \end{bmatrix}$$

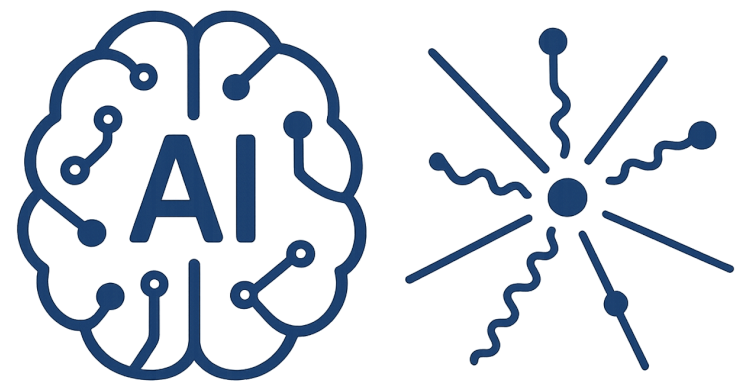
$$X = \begin{bmatrix} X_1 \\ X_2 \\ \vdots \\ X_N \end{bmatrix} \quad W_k = \begin{bmatrix} \vec{w}_{k1} \\ \vec{w}_{k2} \\ \vec{w}_{k3} \end{bmatrix}$$

$$Q = XW_q^T = \begin{bmatrix} \vec{Q}_1 \\ \vec{Q}_2 \\ \vdots \\ \vec{Q}_N \end{bmatrix}$$

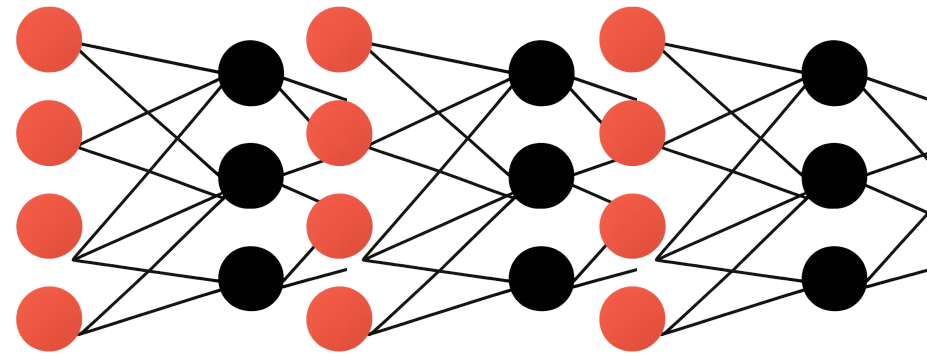
$$QK^T = \begin{bmatrix} \vec{Q}_1 \\ \vec{Q}_2 \\ \vdots \\ \vec{Q}_N \end{bmatrix} \begin{bmatrix} \vec{K}_1 & \vec{K}_2 & \dots & \vec{K}_N \end{bmatrix} = \begin{bmatrix} \vec{Q}_1 \cdot \vec{K}_1 & \vec{Q}_1 \cdot \vec{K}_2 & \dots \\ \vec{Q}_2 \cdot \vec{K}_1 & \vec{Q}_2 \cdot \vec{K}_2 & \dots \\ \vdots & \vdots & \dots \\ \vec{Q}_N \cdot \vec{K}_1 & \vec{Q}_N \cdot \vec{K}_2 & \dots \end{bmatrix}$$

$$X_i = [\phi_i, \eta_i, pT_i, E_i]$$

$$V = XW_v^T = \begin{bmatrix} \vec{V}_1 \\ \vec{V}_2 \\ \vdots \\ \vec{V}_N \end{bmatrix}$$



## Training methods

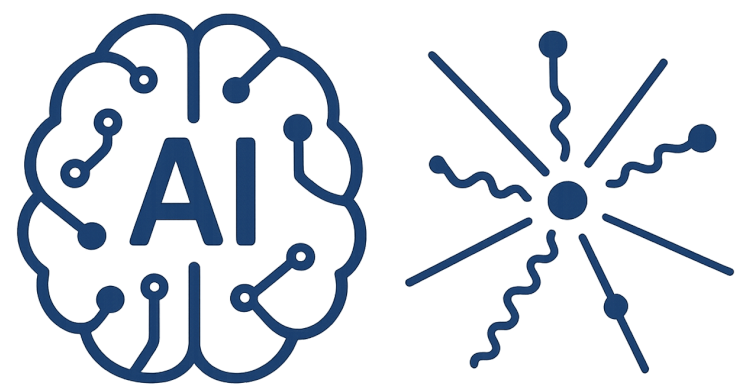


Masked particles

Generate a whole jet

Contrast the object with symmetries (Ex : boost)

Pre-Train model on a cheap simulation data set  
and finetune on a percent of real data  
needs calibration



# AI-Driven HEP 19

## FM4HEP

### Panda: Self-distillation of Reusable Sensor-level Representations for High Energy Physics

Samuel Young<sup>1,2</sup> Kazuhiro Terao<sup>2</sup>

<sup>1</sup>Stanford University    <sup>2</sup>SLAC National Accelerator Laboratory

<https://github.com/DeepLearnPhysics/Panda>

#### Physics reconstruction problem

- 1) hand engineered pattern recognition algorithms
- 2) task-specific neural networks
- 3) Learn directly from sensor-level representations

